# Graal3D and Torque

# Graal3D and Torque

This part contains 14 notes, from `graal3d/camera.md` through `torque/wire-by-build.md`.

Notes under `graal3d/`, then `torque/`, each in path order.

---

## `graal3d/camera.md`

### Graal3D camera (February 2000 client)

Date: 2026-10-31

Follow-camera behaviour in `Graal3D.exe` (resource stamp 2000-02-05), from a static PE float and control-flow read. No decompile text, no binary addresses. Units are Direct3D Retained Mode world units; one frame tick is the 50 ms `TDXTimer`. Tags: **[bin]** unless marked **[doc]** or **[inf]**.

Related: walking controls and the short camera summary in client-behaviour.md. Collision is separate (collision.md).

#### Status for G3D-2

| Need | Value | Tag |
|---|---|---|
| Follow distance | **220** behind the player | [bin] |
| Camera height / look height | follow target **80** above the player on Y | [bin] |
| Field of view | library field **0.5**; opening **2×atan(0.5)** ≈ **53.13°** on either axis of that square window | [doc] for 0.5; [inf] for the degrees; [bin] that the exe does not set it |
| Follow rule | see below | [bin] |

#### Follow rule (Numpad 0 toggles follow on/off)

When the follow flag is on, each tick:

1. Measure the horizontal distance `d` from the camera frame to the player.
2. Close the gap by factor `((d − 220) / 220)² / 10`, with the sign of `d − 220` (move toward the 220-unit target). Skip the move if the factor is 0 or ≥ 100. **[bin]**
3. On Y, move by `(player_y + 80 − camera_y) / 10` (target **80** above the player). **[bin]**
4. Turn the camera toward the player by up to **±5 degrees** per tick (more when the player is behind the camera). **[bin]**
5. Aim at the player (`LookAt`), then pitch by **−0.15 radians** (~−8.6°) about the camera's local X axis. **[bin]**

When follow is off, the follow translation path is skipped (the camera is left where it is for that step). **[bin]**

The player's facing (prop 4) is the angle of the camera-to-player direction, in degrees, sent as `degrees / 2`. **[bin]**

#### Display size (aspect context only)

The main `DXDraw` surface in the form resource is **640×456**. That is the pixel target. The field below stays a square window at any pixel size. **[bin]**

## Field of view

The February 2000 exe leaves the field at the library initial value. The static PE read behind this note found no FOV constant and no `SetPerspective` call. **[bin]** The binary is not in the repo and was not opened again for this section.

Immediately after Direct3DRM creates a viewport, and before any `SetField`, the library state is:

| Quantity | Value | Tag |
|---|---|---|
| Field (`SetField` / `GetField`) | **0.5** | [doc] |
| Window on the front clipping plane | left **−0.5**, right **0.5**, bottom **−0.5**, top **0.5** | [doc] |
| Front clipping plane | **1** | [doc] |
| Projection | perspective | [doc] |
| Opening of that window, along either axis | **2×atan(0.5 / 1) ≈ 53.13°** | [inf] |

**0.5 is the library field parameter.** Degrees in the last row are arithmetic on that window: `SetField` builds a centered square on the front clipping plane, half the window is 0.5, and the front plane sits at distance 1, so each axis opens by `2×atan(0.5)`. X and Y use that same angle. A modern camera that takes a 53.13° vertical angle and then widens X to the 640×456 pixel ratio is a different window; this note does not give that widened angle. The retained-mode viewport uses one scale on X and Y, taken from the longer pixel side (same conformance test). That scale was not turned into a second degree figure.

Two public texts agree on the field value. The DirectX SDK retained-mode viewport reference says the initial field is 0.5 (`SetField` remarks) and the initial front plane is 1 (`SetFront` remarks). `SetPlane` is an arbitrary window on that front plane, and `SetField` is the centered form of the same window. Wine's `d3drm` conformance test (`dlls/d3drm/tests/d3drm.c`, `test_Viewport`) expects the same state right after init: field 0.5, plane ±0.5 on all four sides, front 1, back 100, projection perspective. That test builds the viewport from a 640×480 window and still records the square plane. The test is written to match Microsoft's `d3drm`. This note did not re-run it.

The back clipping plane of this client stays open. The library default back plane is 100. **[doc]** A later public DelphiX (unDelphiX `Source/DXDraws.pas`, retained-mode init) calls `CreateViewport` and `SetBack(5000)` and does not call `SetField` or `SetPlane`. That fork starts after 2004, so it is not the unit linked in February 2000, and this client's far plane was not re-read. The same init leaves the field alone. A field override compiled into `Graal3D.exe` would have been a float constant; the PE read found none.

## Still needed

- ~~FOV from a library default or a lab measurement.~~ **Closed** above: field **0.5**, opening ≈ 53.13°.
- **Far plane stays open.** The library's initial back plane is 100. **[doc]** A later public DelphiX (unDelphiX `Source/DXDraws.pas`) calls `SetBack(5000)` during retained-mode init and leaves the field alone. That fork starts after 2004. The static PE read that found no field constant and no `SetPerspective` did not re-read `SetBack`. The binary is not in the repo, and no other note gives a back-plane number for the February 2000 exe. 100 and 5000 are both sourced. Neither is tagged as this client's far plane.
- Live confirmation of the 220 / 80 / 5° / −0.15 rad figures, and of this field, under Wine with native `d3drm.dll` (Wine's RM stubs break camera frames; see lab-2026-10-30.md).

**`graal3d/client-behaviour.md`**

## Graal3D client behaviour

Date: 2026-10-30

How the February 2000 `Graal3D.exe` moves the player, drives the camera and shows chat, read from the binary for a per-version client profile. **[bin]** unless tagged. Units are Direct3D RM world units; one frame tick is the 50 ms `TDXTimer`.

### Controls

| Key | Action |
|---|---|
| Numpad 8 / 2 | Forward / back, relative to the camera's view direction |
| Numpad 4 / 6 | Step left / right (strafe), at right angles to the view direction |
| Numpad 5 | Jump |
| Numpad 0 | Toggles the camera mode flag (follow camera on or off) |
| Enter | Sends the chat line |
| Ctrl+Enter | Fullscreen on or off |
| F1 / F2 | Hardware options / Player look |
| Esc | Quit |

Movement keys are read with `GetAsyncKeyState`, and only while the chat line does not have focus and the window is active.

### Walking and running

- Walking speed is **8 units per tick** (160 per second).
- After **25 ticks** (1.25 s) of walking without stopping, speed becomes ×1.5 (12 per tick, 240 per second). Stopping resets the count.
- Holding only "back" walks backwards along the direction the player faced when the key went down, instead of following the camera.
- The world is ±4000 units on x and z. A step that would leave it is dropped. This matches the wire range of props 1–3 (`(v − 8000) / 2`, so −4000 to 4095.5 in half-unit steps).
- Collision tests the new position against every face of the level mesh (the faces are loaded with the level). Numbers for a `Collider` (radius 5, step height 30, slope and wall rules, probe order) are in collision.md.
- While walking, the body bobs: y gets `|sin(3.14 × 2 × (anim − 1) / 40)| × 3` added (the program uses 3.14, not π).

### Animation counter (prop 5)

Prop 5 is a tick counter for the walk and jump poses, not a pose id.

- Starting to walk sets it to 0 or 20 at random, so two players do not step in time.
- A jump sets it to 40.
- Where the counter is advanced each tick was not traced; that standing still freezes it is **[inf]**.

### Jumping

- The jump is a fixed table of upward speeds per tick: **10, 10, 9, 8, 6, 3, 1** (47 units up in 7 ticks).

- At the top, the same table is walked back down with negative speeds (−1, −3, −6, −8, −9, −10), and then −10 per tick until the player lands.
- **Letting go of the key early** cuts the climb: the fall starts at once from the top of the table, so a tap gives a short hop.
- A new jump needs the key released and pressed again.

### Camera

Full write-up: camera.md.

- A follow camera stays about **220 units** behind the player. It closes the gap by a factor of `((d − 220) / 220)² / 10` per tick, with the sign of `d − 220`.
- The Y follow target is **80** above the player (`(player_y + 80 − camera_y) / 10` per tick).
- It turns towards the player by up to **5 degrees per tick**, more when the player is behind it, then pitches **−0.15 rad**.
- Field of view is the Direct3DRM initial field **0.5** (square window, opening about 53.13° on either axis). This exe does not set it. See camera.md. **[doc][bin]**
- The player's facing (prop 4) is the angle of the camera-to-player direction, in degrees, sent as `degrees / 2`.

### Chat

There are **no chat bubbles** in Graal3D.

- A chat line (prop 13, or a UDP chat datagram) is added to a **chat log** as `nickname: text`.
- The log is drawn at the bottom left of the 3D view: x = 5, newest line at y ≈ 451, older lines above it.
- The font is "Dialog", 14 px, white.
- Each line stays for **10 seconds** (200 ticks at the default 50 ms timer) and is then removed.
- Your own chat goes to UDP peers right away and to the server in the next props diff. A line longer than 200 characters is cut and ends in `...`.

### Look

- The player is eight meshes: body, head, two arms, two hands, two feet.
- Props 6–9 name the body and head meshes and textures; props 10, 11 and 17 colour the arms, feet and hands.
- The defaults are in formats.md.
- The "Player look" window edits them, and they are saved in `graal3dconfig.txt`.

---

## `graal3d/collision.md`

## Graal3D collision (February 2000 client)

Date: 2026-10-31

Per-face movement test in `Graal3D.exe` (resource stamp 2000-02-05), recovered by static float and control-flow reading of the PE only. No decompile text, no binary addresses. Units are Direct3D Retained Mode world units. Tags: **[bin]** unless marked **[inf]**.

Walking, running and the world clamp that feed this test are in client-behaviour.md. A Wine lab cannot exercise this path without Microsoft's `d3drm.dll` (Wine's `GetFaces` is a stub; see lab-2026-10-30.md).

## What the Collider needs

| Quantity | Value | Tag |
|---|---|---|
| Horizontal body half-extent (radius) | **5** | [bin] |
| Upper-body probe height | **+35** above the feet position | [bin] |
| Upper-body lateral probe half-span | **10** (probes at 0, +10, −10 on the strafe axis only) | [bin] |
| Max step height (onto a walkable face) | **30** | [bin] |
| Walkable face normal | upward component > **0** | [bin] |
| Feet-mesh attach slope | upward normal > **0.2** (about 78.5° from horizontal); steeper walkable faces still snap the point but skip the mesh attach | [bin] |
| Wall “already clear” distance | current signed distance ≥ **3** → no wall hit | [bin] |
| Wall “started inside” distance | previous signed distance < **−5** → no wall hit | [bin] |
| World clamp (x and z) | **±4000**; a step that would leave is dropped | [bin] |

Sliding: on a hit the client projects the test point onto the face plane along the face normal (subtract `dist × normal` from the point). **[bin]**

## Order of tests (one attempted step)

1. Build the candidate position from the walk/run/jump delta (see client-behaviour.md).
2. Load every face of the level mesh (the face list filled at level load).
3. Transform face vertices into world space.
4. Run the **body probes** below against every face.
5. Reject the step if a probe reports blocked; otherwise keep the (possibly slid) position.
6. Drop the step if the resulting x or z is outside ±4000.

## Body probes

From the candidate feet position the client tests these points, in order. A probe that returns blocked stops the rest for that step.

**Ground / step probes** (flag “floor”): four points in the horizontal plane of the feet, offset by **±5** along the facing axis and by **±5** along the strafe axis (a plus shape, not a filled disc). **[bin]**

**Upper probes** (flag “wall only”): raise Y by **35**, then test at lateral offsets **0**, **+10** and **−10** on the strafe axis only. These probes do not run the floor/step path; they only participate in the wall/ceiling path. **[bin]**

## Per-face test

For each face the client has the face vertices and a face normal.

#### Over-face test

Before using a face as a floor or wall, the probe’s **(x, z)** must lie inside the face’s horizontal polygon: a winding / cross-product test on the projected edges (both edge orders are tried). If the point is outside, the face is skipped. **[bin]**

#### Walkable faces (normal Y > 0)

Only the ground probes use this path.

1. Signed distance of the probe to the face plane: `dot(probe − vertex0, normal)`.
2. If distance > **30**: treat as landing on this face (place the player on the plane, set the landed / not-jumping flags). Used when the feet are far above the face (for example after a jump). **[bin]**
3. If distance < **−30**: fail this face (step too high; the ledge is more than 30 below the feet along the normal). **[bin]**
4. Otherwise (−30 ≤ dist ≤ 30): **slide** the probe onto the plane. If normal Y > **0.2**, also attach / orient the player mesh to the face; if 0 < normal Y ≤ 0.2 the point still slides but the mesh attach is skipped. **[bin]**

So the **step height is 30**: a walkable ledge up to 30 units above the feet can be stepped onto; taller ledges are blocked. **[bin]**

#### Walls and ceilings (normal Y ≤ 0)

Used by ground and upper probes.

1. Signed distance of the **new** probe position to the plane, and of the **previous** position.
2. Drop the face if the new distance is ≥ **3** (still clear of the wall). **[bin]**
3. Drop the face if the previous distance is < **−5** (started already deep inside). **[bin]**
4. Drop the face if the move vector's length squared is ≤ **0**. **[bin]**
5. Otherwise **slide** the new point onto the plane (same projection as above). **[bin]**

### What this does not settle

- Exact player capsule vs the plus-shaped probe set (a disc of radius 5 would be an approximation, not what the binary runs). **[inf]** if a reimplementation substitutes a disc.
- Floor friction, ground stickiness beyond the slide-to-plane rule, or separate ceiling thickness. Nothing else showed up in the float set of this routine. **[bin]** for absence in this path.
- Live confirmation under Wine still needs native `d3drm.dll`. **[inf]** until a lab run.

---

## `graal3d/formats.md`

## Graal3D files

Date: 2026-10-21

### `graal3dconfig.txt` (client)

`key=value` lines, read at start and written back when settings change:

| Key | Shipped value | Meaning |
|---|---|---|
| `serveraddr` | `<graal3d-default-host>` | Server host or IP (the connect window default is the loopback address) |
| `serverport` | `14901` | TCP port |
| `fixedport` | `false` | Bind the UDP socket to `fixedudpport` instead of a random port |
| `fixedudpport` | `14898` | |

| Key | Shipped value | Meaning |
|---|---|---|
| `noudp` | `false` | No UDP socket. Position and chat then go only through the server. |
| `bodyx` | `body.x` | Body mesh (prop 6) |
| `headx` | `head.x` | Head mesh (prop 7) |
| `bodytex` | `body.bmp` | Body texture (prop 8) |
| `armcolor` | `0xFF0000` | Prop 10 |
| `shoecolor` | `0x800000` | Prop 11 |
| `handcolor` | `0xFDCC47` | Prop 17 |

The reader also accepts `accountname` and `password`.

The client resolves host names once per run and keeps the name → address pairs in a small cache.

### Registry (client)

`HKCU\Software\Graal3D`, section `Client`: `nickname`, `accountname`, `password`.

The password is stored as 48 printable characters, two per byte, giving 24 bytes. Those are three DES blocks under a fixed 8-character key built into the exe. Decrypted, byte 0 is the length and the password follows. This is obfuscation, not protection. It uses the same `DesCipher` class that later 2D clients use to hide their list-server host.

### `serverconfig.txt` (server)

`serverport=14901`.

### `accounts.txt` (server)

One account per `\n`-terminated line, in the protocol's `+32` text codec. Confirmed byte for byte in the lab, and a hand-written file is accepted. **[cap]**

```
str(name) str(password) str(saved props) c(flags) c(0) c(0) c(0) c(0) c(0) c(0) c(0)
```

For example, a fresh administrator account `alice` / `pw1` is `%alice#pw1 &` followed by seven spaces and `\n`.

- **saved props** is the player's prop run (the same encoding as packet 1): props 1–14, 16 and 17, no 0 or 15. It is empty until the player first leaves. It is written back when a bound player leaves or is edited, and applied on the next login (positions, chat text and nickname included).
- **flags**: bit 0 = Load only, bits 1–3 = rights level (0 No rights, 1 Graalian Police, 2 Trusted Admin, 3 Administrator), bit 4 = Blocked.
- **The seven trailing bytes** are the empty count-prefixed lists inherited from the 2D account record. The reader accepts lists there, but no Graal3D prop or packet fills them.
- The server rewrites the file whenever an account changes, not only at shutdown.

### `guilds\guild<name>.txt` (server)

A nickname `nick (Name)` keeps its `(Name)` tag only if this file exists and lists the player's account (one per line). This works the same as in the 2D server.

### Levels (`levels\*.x` on the server)

A level is a **DirectX retained-mode `.x` text file** (`xof 0302txt 0064`). `startlevel.x` is the server's start level and `level1.x` is the client's default. The two shipped files hold the same scene and differ only in whitespace (`level1.x` has CRLF line ends and trailing spaces, `startlevel.x` has LF line ends):

- One 139-vertex mesh named `obj`, with a material list and texture coordinates.
- Nine materials: `grasstex`, `waytex`, `stonestex`, `walltex`, `towertex`, `platformtex`, `cloudstex`, `plainblue`, `plaingray`.
- Textures referenced by file name: `grass.bmp`, `way.bmp`, `stones.bmp`, `wall.bmp`, `tower.bmp`, `platform.bmp`, `clouds.bmp`.

The server caches each level's text and sends it inline in packet 0 (`\n` written as §). The client saves the text to disk under the level name and loads it with Direct3D RM. **Textures are not transferred**: the BMPs a level names must already be in the client folder. There is no tile layer, NPC or link data. A level change is a change of prop 14.

### Player model (client)

Eight `.x` meshes (`xof 0303txt 0032`), each with one mesh, normals, texture coordinates and a material:

| File | Notes |
|---|---|
| `body.x` | Textured with `body.bmp` (128×128, 24-bit; `body1.bmp` is an alternative) |
| `head.x` | Three materials, textured with `man.bmp` (64×64) |
| `armright.x`, `armleft.x` | Coloured by prop 10 |
| `handright.x`, `handleft.x` | Coloured by prop 17 |
| `footright.x`, `footleft.x` | Coloured by prop 11 |

The body and head mesh and texture names travel as props 6–9, so a player can point them at other files present on every client. Colours are 24-bit RGB set as material colours at runtime. Prop 5 is passed to the model's animation object; it is most likely the animation frame.

### Textures

Plain Windows BMPs: 24-bit for `body.bmp`, `body1.bmp`, `grass.bmp`, `tower.bmp`, `man.bmp` and `logo.bmp`; 8-bit RLE for `clouds.bmp` (256×256), `platform.bmp`, `stones.bmp`, `wall.bmp` and `way.bmp`. Most are 128×128.

---

## `graal3d/lab-2026-10-30.md`

## Graal3D live lab (2026-10-30)

The real `GServer.exe` (MD5 `aab1a34a1f196d4f4fddf12112ccbf52`) and `Graal3D.exe` (MD5 `305c985a020cec6ab8d81ac95d447343`) from `graal3d.7z`, run under Wine 9.0 (32-bit prefix) with Xvfb in the cloud container. The results are folded into protocol.md and formats.md with a **[cap]** tag.

### Setup

- **Server.** `GServer.exe` starts with an "Options / Start on port" dialog (default 14901). After OK it runs as a tray program with a small status window: "Server started at port 14901", "Server name: <host name>",

"Current session count", and the buttons View players, Edit accounts, Update levels and Disconnect all RCs.
- **Levels.** The archive ships no `levels\` folder. The server needs `levels\startlevel.x`; copying the shipped `startlevel.x` there is enough.
- **Accounts.** The archive ships no `accounts.txt`, so every login fails until accounts exist. They can be made in Edit accounts (a tree with the folders Guest Accounts, Regular Accounts, Graalian Police, Trusted Admins, Administrators) or written by hand in the format in `formats.md`. The account editor holds: name, password, Load only, Blocked, Last IP, the four rights levels, and the player's level, view angle (default 270) and x/y/z, plus the look fields.
- **Traffic.** A small TCP proxy between client and server inflated every frame and logged each message both ways. UDP was recorded with tcpdump.
- **Clients.** The real client for login, props and the level transfer. Scripted clients (Python, the same codec) for everything in-world, because of the Wine limit below.

### The Wine limit

Graal3D draws with Direct3D **Retained Mode** (`d3drm.dll`). Wine's own `d3drm` is mostly stubs: frame `SetPosition`, `GetPosition` and `LookAt`, and mesh builder `GetFaces`, return without doing anything. As a result:

1. The first frame tick after the connect window crashes in the movement routine (access violation reading address 0), before the client flushes its login. Skipping that routine lets the login out.
2. Loading the level then stops at `GetFaces`, so the player frames are never made. Skipping the face list (it only feeds collision) lets the level load.
3. Loading the eight player meshes still fails, so other players are never created on screen. Positions read back from Wine are garbage, so moving would raise "Invalid floating point operation".

The lab therefore patched two places in the running process (not the file: the client checks its own exe and says "The Graal execution file is corrupted" if the file changes). With that, the real client connects, logs in, sends its look, nickname and UDP port, receives its props and the level, and saves the level file. It cannot show the world or other players, and it cannot be walked around.

**To go further,** the client needs Microsoft's own `d3drm.dll` (it is in `dxnt.cab` inside the DirectX February 2010 redistributable; winetricks has a `d3drm` verb for it). The container's network policy blocked that download. With the native DLL in the prefix, the same lab could capture movement, UDP position and chat between two real clients.

### What was confirmed

- Accept: `6` + `c(0)` before login. Login reject texts and the close.
- Login order, own props, the full props exchange, level name + text + leader, prop 50.
- The UDP relay rule: positions and chat go over TCP only when one side has no UDP port.
- Nickname guild-tag stripping and when the `3` echo is sent.
- Prop 14 from a client does not move the player.
- Saved props on the next login, and the saved nickname relayed first.
- Load-only accounts can log in many times; normal accounts give "The account is already in use."
- Disconnect: prop 51 to players, 56 to admins.
- Admin login (id 1): the account list (50), records (54), player joined/left (55/56), player props (59), message to all (8), kick (5).
- `accounts.txt`: exact layout, and a hand-written file is accepted.

A reference server written from the notes (reference-server/) gave the **same message stream** as the original for the scripted two-player session (login, moves with and without UDP, UDP hand-over, chat, level prop, leave, and a second login with saved props). The real client logged into it and received a byte-identical level file.

### Not confirmed

- UDP between two real clients, and how the client applies a peer's datagram (needs the native `d3drm.dll`).
- Admin commands 51–54, 59 (reply 57), 60 and 62 beyond the replies listed above.
- The "Disconnect all RCs" button and the player-list kick (packet 16) were not pressed.

---

## `graal3d/protocol.md`

## Graal3D protocol

Date: 2026-10-21

The Graal3D client and server use TCP port 14901, plus UDP from client to client.

Updated 2026-10-30 with a live lab: the real `GServer.exe` and `Graal3D.exe` under Wine, a logging proxy between them, and scripted clients. Tags: **[cap]** seen on the wire, **[bin]** read from the binaries only, **[inf]** inferred. Untagged lines from the first pass are static reads that the captures did not contradict. See lab-2026-10-30.md for the setup and its limits, and reference-server/ for a server written from these notes that the real client logs into.

### Encoding

The same text codec as 2D Graal (see `protocol/PROTOCOL-CHEATSHEET.md`):

- `c(v)` is one byte, `v + 32`.
- `sN(v)` is an N-byte unsigned integer, 7 bits per byte, most significant first, each byte `+ 32`.
- `str(s)` is `c(len)` then the bytes.

### TCP frames

```
[u16 BE length] [zlib block]
```

The zlib block holds one or more messages, each `c(id)` + payload + `\n`. Both sides queue messages and compress when the queue passes 4096 bytes or on the frame-tick flush. A client drops the link if more than 256 KiB of compressed output is pending. The client sends in 256-byte slices. The receive side inflates into a buffer of up to 256,000 bytes.

There is no key byte or scramble. A message must not contain `\n`; text with line breaks (level files, admin messages) replaces each `\n` with § (0xA7). **[cap]**

The client puts its login line and its first props line in the same frame. The server answers each event with its own frame (one for the accept, one per login reply, one for the level text), so a server can flush per event and does not need the 4096-byte batching to be compatible. **[cap]**

### Login

The first line from the client:

```
c(0)  "GR3DV002"  str(account)  str(password)
```

The server reads the version at bytes 2 to 9. `GR3DV001` gets packet 5 "You need at least Graal3D beta2…". If an account is missing, in use or disabled, the server sends 5 with the matching text. A line with id 1 and the same layout is an admin login. It needs an account with admin level ≥ 1, puts the session on the admin list, and gets the account list (50) and player list (55).

On TCP accept, before login, the server sends `6` with `c(0)` (the bytes `&` after inflating, frame length 11). **[cap]**

Login reject texts seen live: "An account with the name and password you typed in doesn't exist." and "The account is already in use." The server sends 5 and closes. **[cap]** It still parses the rest of the frame first: a rejected client's props line is applied to a throwaway player, so a nickname with a `(guild)` tag gets a `3` echo after the 5, and other players can briefly see that throwaway player's props. A server does not need to copy this. **[cap]**

On a player login the server, in this order **[cap]**: 1. Gives the player the lowest free id from 1 to 200, links the account and applies the props saved in the account record (see `formats.md`). If the saved nickname is not empty, applying it relays `2` + prop 12 to every other player at once, so that line arrives before the full props. 2. Sends `3` with props 0–5 (id, x, y, z, angle, animation). 3. For each other player, sends `2` with all of props 0–17 to the new player, and the new player's props 0–17 to them. 4. Sends the player to their level (below): `1`, `0`, then `2` + prop 50 = 1 both ways for everyone already there, or `4` if the level is empty. 5. Tells admin sessions with `55`.

Then the client's first props line is handled like any other (below), which is why the others get prop 12 and prop 15 again on their own and then once more inside the level relay.

A login of an account that is not "load only" while it is in use is rejected. Load-only accounts (the guest folder in the server UI) can log in many times at once. **[cap]**

### Server → client

| Id | Name used here | Payload | Client action |
|---|---|---|---|
| 0 | LEVEL | level file text, `\n` as §, or empty | Saves the text under the current level name and loads it as a `.x` mesh. Empty means "you already have it". |
| 1 | LEVELNAME | rest = file name | Stored as the current level name. |
| 2 | OTHERPLPROPS | `c(player id)` + props | Creates the player on first sight, then applies the props. |
| 3 | PLAYERPROPS | props | Applies to the local player. |
| 4 | (leader) | empty | Sent to the first player in a level, and to the next one when the first leaves. The client sets a flag that packet 0 clears; nothing in the client reads that flag. **[bin][cap]** |
| 5 | DISCMESSAGE | rest = text | Stored. Shown when the socket closes ("You have been disconnected from the server." if empty). |
| 6 | (session) | `c(v)` | Stores `v` in a global that the connect routine resets to 0. Nothing reads it. The server sends 0 on accept. **[bin][cap]** |
| 7 | FILESENDFAILED | rest = file name | Not handled by the client. |
| 8 | ADMINMESSAGE | rest, § = line break | Shown in an "Administrator message" box. |

| Id | Name used here | Payload | Client action |
|---|---|---|---|
| 16 | (2D discmessage) | text | Not handled. Sent only by the server's player-list kick button. |
| 50–59 | admin replies | see below | Admin sessions only. |
| 100 | RAWDATA | `s3(len)` | The client flags the next line as raw and then discards it. |
| 101 | FILE | `str(name)` + bytes | Server answer to request 2. Unused by this client. |

### Client → server

| Id | Payload | Server action |
|---|---|---|
| 0 | login (above) | |
| 1 | props | Applies, then relays (below). |
| 2 | file name | Looks in `levels\`. Sends 100 + 101 if the file is under 64,500 bytes, else 7. Never sent by the shipped client. |

Admin commands (51–64) are listed at the end.

### Player properties

The same prop ids are used in both directions and inside UDP. A props payload is a run of `c(prop id)` + value with no separators.

| Id | Value | Field | Default | Client sends |
|---|---|---|---|---|
| 0 | `c` | player id (1–200) | – | no |
| 1 | `s2` | x: `(v − 8000) / 2` | 0 | yes |
| 2 | `s2` | y, same scale | 0 | yes |
| 3 | `s2` | z, same scale | 0 | yes |
| 4 | `c` | angle: `v × 2` degrees | 0 | yes |
| 5 | `s2` | animation frame (likely) | 0 | yes |
| 6 | `str` | body model | `body.x` | yes |
| 7 | `str` | head model | `head.x` | yes |
| 8 | `str` | body texture (reloads on change) | `body.bmp` | yes |
| 9 | `str` | head texture | `man.bmp` | yes |
| 10 | 4 bytes, 6 bits each | arm colour, 24-bit RGB | `FF0000` | yes |
| 11 | 4 × 6 bits | shoe colour | `800000` | yes |
| 12 | `str` (≤ 200) | nickname, optional `(guild)` | – | yes |
| 13 | `str` (≤ 200) | chat text | – | yes |
| 14 | `str` | level file | `level1.x` (client), `startlevel.x` (server) | no (a client that sends it only changes the stored value, see below) |
| 15 | `s3` | UDP port, 0 = none | 0 | yes (its own bound port) |
| 16 | 5 bytes: 4 bits then 4 × 7 | IPv4 address: the in-memory `in_addr` word read as a little-endian integer, so the loopback address is 0x0100007F and goes out as `0 (  \x9f` **[cap]** | – | no (the server fills in the TCP peer address) |

| Id | Value | Field | Default | Client sends |
|---|---|---|---|---|
| 17 | 4 × 6 bits | hand colour | `FDCC47` | yes |
| 50 | `c` | 1 = this player is in your level (add as UDP peer), 0 = not | – | server only |
| 51 | none | player left: remove | – | server only |

The colour props map to mesh pairs: 10 to the two arms, 17 to the two hands, 11 to the two feet. The pairing comes from the mesh slots each prop recolours plus the config key names.

Each tick the client re-encodes all props it sends, compares them with the last tick, and sends the changed ones as one `1` message. Props 1–5 are also compared separately for UDP (below). The first send after connecting carries the full look (6–12 and 15).

#### Server relay rules

- On prop 12, the server splits `nick (guild)`. It keeps the tag only if `guilds\guild<guild>.txt` lists the account. If the text had a tag, it echoes the result back with `3` (`Alice (Admins)` with no guild file came back as `3` + `Alice`; a plain `Bob` got no echo). The nickname then goes to every player (2) and every admin session (59). **[cap]**
- On prop 15, the server sends `2` + prop 15 to every player. **[cap]**
- Prop 14 from a client is stored and relayed to the players in the player's current level, but **it does not move the player**: no `1`/`0` comes back and the level membership stays the same. Only the server moves players (on login, and when an admin edits an online player with 60). **[cap][bin]**
- Prop 16 from a client is ignored while the player has a live socket.
- All changed props in the line, including 12 and 15, then go as one `2` line to the players in the same level. **Props 1–5 and 13 are left out when both players have a UDP port**, because those go peer to peer; if nothing is left, nothing is sent. When either side has no UDP, the observer gets everything. **[cap]** Each observer has its own dirty set, so a player who was skipped gets the missing props later.

Example from the lab, A with UDP and B without: A's `1` with position, animation and chat reached B as `2 + ! ! (…)` with props 1–5 and 13. After B announced a UDP port, A's next position and chat produced nothing on TCP. **[cap]**

#### Level change

1. The server finds the level in its cache, or loads it from `levels\<name>`.
2. It sends `1` (name), then `0` with the file text, or `0` empty if this player was already sent that level. The "already sent" list is per player, not per account: a second login gets the full text again. **[cap]** The client writes the text to a file with that name in its own folder (the lab copy was byte-identical to the server's LF-only `startlevel.x`). **[cap]**
3. For each player already in the level, it sends both sides `2` with `50 = 1` and the props that player has not yet seen.
4. If the player is alone in the level, it sends `4`.
5. It tells admin sessions with `59`.

On leaving a level the others get `50 = 0`. On disconnect every player gets `c(id)` + prop 51, and admin sessions get `56` + `c(id)`. **[cap]**

### UDP (client to client)

The client binds one UDP socket (port 0, or the fixed port from the config) unless `noudp` is set. It tells the server the bound port in prop 15. The server adds the peer's IP as prop 16. The client keeps a peer list of players flagged with `50 = 1` that have both a UDP port and an address.

Datagrams have no length prefix and no compression:

| First byte | Body | Meaning |
|---|---|---|
| `c(0)` | the values of props 1–5 back to back, without prop ids (9 bytes) | position update; ignored if 8 bytes or fewer |
| `c(1)` | chat text, up to 200 bytes (longer is cut and given `...`) | chat line |

A receiver accepts a datagram only if its source address and port match a known player, and applies it to that player. The sender sends one copy to each peer: position only when props 1–5 changed since the last tick, and chat right after Enter. The chat also goes to the server as prop 13 in the next props diff.

### Admin commands

The server keeps these from the 2D GServer. Each needs a minimum admin level (0 none, 1 and 2 staff, 3 full). An admin can only act on accounts at or below their own level, unless they are level 3.

Account ids are a 28-bit number sent as 4 bytes, 7 bits each.

| Id in | Payload | Effect / reply |
|---|---|---|
| 51 | account id | Reply 54: account record. |
| 52 | account id, `str` name, `str` password, `str` props, `c` flags | Edit account. → 51 (changed) and 52 (renamed) to admins. |
| 53 | account record | Add account. → 50 to admins. |
| 54 | account id | Delete account. → 53 to admins. |
| 59 | `c(player id)` | Reply 57: that player's account. |
| 60 | `c(player id)` + account record | Edit an online player's account. |
| 61 | `c(player id)` | Kick: the player gets 5 "The server administrator has disconnected you." and the others get prop 51. **[cap]** |
| 62 | `s2(count)` + `str` names | Reload these levels from disk. |
| 63 | text | Message to all players (8) as `Administrator <account>:§<text>`. **[cap]** |
| 64 | `c(player id)` + text | Message to one player (8). |

Server → admin: 50 account added, 51 account changed, 52 account renamed, 53 account deleted, 54 account record, 55 player joined, 56 player left, 57 player's account, 58 player's account renamed, 59 player props (level, nickname). The 55/56 ids match the 2D add/remove-player ids. Names for the others are descriptive, not recovered.

Layouts seen live **[cap]**:

| Id | Layout | Example |
|---|---|---|
| 50 | account id (4 × 7 bits), `c(folder)`, name to end of line | `" - alice`. The folder byte is 1 for a normal account plus 4 × admin level, 0 for a load-only account (alice, level 3: 13; bob: 1; guest: 0). The admin login sends one 50 per account. **[inf]** for the formula, fitted to three accounts. |

| Id | Layout | Example |
|---|---|---|
| 54 | account id, `str(password field)`, `str(props)`, `c(flags)` | The password field came back empty. Props are the saved prop run (1–14, 16, 17; no 0 or 15). Flags as in `accounts.txt`. |
| 51 | account id, `c(flags)` | Sent when a bound player leaves and the record is saved. |
| 55 | `c(player id)`, `str(account)`, prop 14 | `! #bob .,startlevel.x` |
| 56 | `c(player id)` | |
| 59 | `c(player id)` + props | Mirrors what players see: the nickname, the full props on login, prop 14 on level entry. |

Account flags byte (also in `accounts.txt`): bit 0 = load only (the server UI's "Load only" box: the session is not tied to the record, props are not saved back, and many logins may share it), bits 1–3 = admin level, bit 4 = blocked. **[cap]** (Alice, Administrator: `&` = 6; guest, Load only: `!` = 1.) The server UI names the levels No rights (0), Graalian Police (1), Trusted Admin (2) and Administrator (3).

---

### `graal3d/README.md`

## Graal3D (February 2000)

Date: 2026-10-21

Graal3D is a separate 3D side project shipped as `graal3d.7z`: a client, `Graal3D.exe`, and its own server, `GServer.exe`. Both carry resource stamps from 2000-02-05, between the 2D releases V1.32 (`GSERV018`) and V1.33 (the first `GNW` token). These notes come from static reads of both binaries (RTTI, form resources, strings), a decompiler pass, and a live lab on 2026-10-30. No decompiler output is reproduced here.

**Does it use a gserver?** Yes, its own: `GServer.exe` from the same archive (TCP 14901). It does not talk to the 2D GServer or the list server. Players also exchange position and chat directly over UDP once the server has told each client the others' addresses.

- protocol.md: framing, login, packet ids, the player property table, peer-to-peer UDP, admin commands.
- formats.md: config files, the registry password, `accounts.txt`, levels and models.
- client-behaviour.md: controls, walking, running, jumping, camera and the chat log, for a per-version profile.
- collision.md: per-face step test for a `Collider` (radius 5, step height 30, slope and wall rules, probe order), static read of the February 2000 client.
- camera.md: follow distance 220, look height +80, turn ±5°/tick, pitch −0.15 rad; field of view 0.5 (opening about 53.13°). The far plane stays open.
- lab-2026-10-30.md: the live run of both programs under Wine, what it confirmed and its limits.
- reference-server/: a small Python server written from these notes. The real client logs into it, and it matches the original server's messages on the lab session.
- torque-engine-4x.md: the **other** Graal 3D, the Torque-based `Graal3DEngine.dll` of the 4.x clients (2006), and how a 4.x client connects to a 3D server.

### What ships

| File | Notes |
|---|---|
| `Graal3D.exe` | Delphi client. DelphiX `TDXDraw` in Direct3D retained mode (`do3D`, `doRetainedMode`, `doZBuffer`, `doTexture`), 50 ms `TDXTimer`. zlib 1.1.2 linked. |

| File | Notes |
|---|---|
| `GServer.exe` | Delphi server, default TCP port 14901, `levels\` folder, `accounts.txt`. |
| `graal3dconfig.txt` | Client config (server address, port, UDP options, look). |
| `serverconfig.txt` | `serverport=14901`. |
| `level1.x`, `startlevel.x` | Levels as DirectX `.x` text files. |
| `head.x`, `body.x`, `armleft/right.x`, `handleft/right.x`, `footleft/right.x` | The player model, one mesh per body part. |
| `*.bmp` | Textures (`body.bmp`, `body1.bmp`, `man.bmp` for the head, and the level textures). `logo.bmp` is the connect-window image. |

There are no `.graal`/`.nw` loaders, no GANI, no tiles and no NPC scripting in either binary.

### Client windows

| Class | Purpose |
|---|---|
| `TAdventure` | Main window: the 3D view, a chat edit line, the frame timer. Ctrl+Enter toggles fullscreen, Esc quits, F1 and F2 open the two option windows. Enter in the chat line sets the chat text. |
| `TConnectFrame` | Start window: nickname (default `Stefan`), host (default: the loopback address), port `14901`, account and password (both `guest`). |
| `TMoreOptions` | "Hardware options": fixed UDP port (default 14898) and "Don't use UDP". |
| `TEditLook` | "Player look": `bodyx`, `headx`, `bodytex`, arm, shoe and hand colours. |

Each frame tick the client reads the TCP socket, reads the UDP socket, diffs its own player properties and queues the changes, then flushes the TCP send buffer.

### Server windows

| Class | Purpose |
|---|---|
| `TStartFrame` | Start dialog. |
| `TServerFrame` | Main window: player count, "Edit accounts", "Update levels", tray menu with Show / Shutdown. Caption starts as `Graal3D: 0 player(s)`. |
| `TAccountList`, `TEditAccount` | Account list and editor (name, password, admin level, disabled flag). |
| `TPlayerStats` | Online player list with message, disconnect and account buttons. |
| `TSendFrame` | Message to one player or to all. |
| `TUpdateLevels` | Names of cached levels to reload from disk. |

The server holds at most 200 players. Player ids are the lowest free slot from 1 to 200.

### How it relates to the 2D Graal of the same time

**The server is the 2D GServer with the game removed.** Every Delphi class in the Graal3D `GServer.exe` also exists in `GServer13.exe` (graal13b, 1999-11-29). The 2D server has 23 more: `TServerNPC`, `TServerBaddy`, `TServerChest`, `TServerHorse`, `TServerLevelLink`, `TBoardModify`, their `*Modify` delta classes, and the rich-edit/printer VCL units. The shared classes keep their names and roles:

- `TServerAccount` is 44 bytes in both servers, and accounts live in `accounts.txt` in both.
- `TServerLevel` shrinks from 8,244 bytes (it held a 64×64 board) to 24 bytes (name, file text, the list of players in it, and the list of players already sent the file).

- `TServerPlayerModify` keeps its job of tracking, per observer, which properties are still unsent.
- The admin forms (`TAccountList`, `TEditAccount`, `TPlayerStats`, `TSendFrame`, `TUpdateLevels`) are carried over.
- The guild tag check (`guilds\guild<name>.txt`) is carried over.

**The wire codec is the 2D codec.** Frames are a 2-byte big-endian length plus a zlib block, one message per line, `id + 32` packet ids, `+32` length-prefixed strings and 7-bit big-endian integers, with a 4096-byte flush. The differences:

- The version string is `GR3DV002` (the server also recognises `GR3DV001` and rejects it with "You need at least Graal3D beta2 to play on this server.").
- There is no key byte and no per-packet scramble.
- Packet ids and the property table are renumbered from scratch: positions are 3D floats, the angle is one byte, and colours are 24-bit.
- Player positions and chat go **peer to peer over UDP**. The server hands each client the other players' UDP endpoints and only relays position and chat over TCP for players that have no UDP. The 2D clients of the time also had `noudp` and `fixedudpport`, but there UDP went to the server.

**The client reuses the 2D client's skeleton.** It has the same class names: `TAdventure` (main form with `DXDraw`/`DXTimer1`), `TAdventureClient` (the socket object), `TAdventureServer` (here a small host-name → address cache entry), `TConnectFrame`, `TMoreOptions`, `TProgramCheck` (the "Graal execution file is corrupted" self-check) and `DesCipher`. The 2D-only parts (level, NPC, weapon and script classes) are gone, and `TPlayer` now holds Direct3D RM frames and meshes.

**Leftovers that show the port.** The server's player-list "disconnect" button sends packet 16, which is the 2D client's disconnect-message id. The Graal3D client ignores 16, so that kick arrives silently. The admin command path (61) sends 5, which the 3D client does show. The server still answers a file request (2) with the 2D raw-data pair 100/101, but the 3D client never sends 2 and drops raw data.

### Open points

- Packets 4 and 6 set client values that nothing reads (checked 2026-10-30). Packet 4 follows 2D "leader" semantics on the server side.
- Movement maths: see client-behaviour.md. Collision: see collision.md. Camera follow numbers and field of view: see camera.md. ~~FOV still **[inf]**.~~ **Closed** there: Direct3DRM field **0.5**, opening `2×atan(0.5)` ≈ 53.13°. The far plane stays open: the library initial back plane is 100 **[doc]**, a later DelphiX fork calls `SetBack(5000)`, and the February 2000 exe was not re-read for `SetBack`.
- No admin client for Graal3D is in the archives. The admin protocol below comes from the server alone.
- The live run (see lab-2026-10-30.md) confirmed the TCP side. UDP between two real clients still needs Microsoft's `d3drm.dll`, which Wine does not provide. Collision was not lab-confirmed for the same reason.

---

## `graal3d/reference-server/README.md`

## Graal3D reference server

`g3dserver.py` is a small server for the February 2000 Graal3D client, written from ../protocol.md. It is plain Python 3 with no dependencies. It exists to show that the notes are enough to host the real client; it is not a copy of `GServer.exe`.

```
python3 g3dserver.py --port 14901 --levels levels --accounts accounts.json
```

- `levels/` holds the `.x` level files. `startlevel.x` is where players start; copy it from the Graal3D archive.
- `accounts.json` maps names to `{"password", "level" (0–3), "loadonly", "disabled"}`. The server saves each bound account's props there when the player leaves. Without `--accounts`, any name and password log in as a load-only account.
- Clients need the level's textures (`grass.bmp` and so on) in their own folder; the protocol never sends them.

### What it does

- Accept and login, including the `GR3DV001` and account error texts.
- Player ids 1–200, saved props, the login order the original uses.
- Level text in packet 0 (empty if already sent to that player), the leader packet, prop 50.
- Props relay with the UDP rule (props 1–5 and 13 skipped when both ends have UDP), nickname guild tags (`guilds/guild<name>.txt` next to the levels folder), prop 15 broadcast.
- Disconnect (prop 51, admin 56).
- Admin login with the account list and player list, message to all (63), message to one (64), kick (61).

Not done: admin account editing (51–54, 59, 60), level reload (62), the file request (2), the 4096-byte send batching. Changing a player's level is only possible from code (`enter_level`), as in the original, where only an admin edit moves a player.

### How it was checked (2026-10-30)

- A scripted two-player session (login, moves with and without UDP, UDP hand-over, chat, prop 14, leave, then a second login with saved props) produced the same messages from this server as from the original `GServer.exe` under Wine, apart from the random UDP ports.
- The real `Graal3D.exe` logged in, received its props and the level, and saved a byte-identical `startlevel.x`. See ../lab-2026-10-30.md for why the real client could not be taken further under Wine.

---

## `graal3d/torque-engine-4x.md`

## Graal 4.x 3D engine (Graal3DEngine.dll)

Date: 2026-10-30

The Windows Graal 4.04r2 client includes a 3D plug-in, `Graal3DEngine.dll` (linked 3 March 2006). It is a cut-down Torque Game Engine (TGE) with Graal changes. This note covers how the 2D client hands over to it, its network protocol to a 3D server, and the files it reads. Static reads only; nothing here has been checked on the wire.

This is not the 2000 standalone Graal3D (TCP 14901, `GR3DV002`); see `protocol.md` in this folder for that one.

Evidence tags as in `protocol/v4x-clients.md`: **[bin]** read from the binary, **[inf]** inferred, **[file]** from the shipped data.

### Overview

- The DLL exports three functions: `loadGraal3DEngine(char *cmdline)`, `runGraal3DEngine()` (one frame) and `endGraal3DEngine()`. It imports 623 symbols from `GraalEngine.dll`. **[bin]**

- The launcher (`Graal4.exe`) loads the DLL **on demand**, the first time the player picks a 3D server. It calls `loadGraal3DEngine("")` and, from then on, `runGraal3DEngine` once per frame. **[bin]**
- Once loaded, the DLL takes over **all** drawing for the engine: it registers the engine's text, bitmap, tile and panel callbacks, then re-initialises the engine. The 2D game then draws through Torque's GUI canvas. **[bin]**
- A 3D session uses **two connections to the same host**:
  - the normal Graal TCP game connection on the listed port;
  - a Torque UDP connection on **port + 1**.

  Both are opened together. **[bin]**
- The Torque console and TorqueScript are gone. Every engine object is exposed to Graal's GS2 script objects instead. There is no TorqueScript compiler, and `.dso` files are never requested. **[bin]**

## Hand-off from 2D

#### Server list entry

A server whose list name starts with `"3 "` is a 3D server. The engine strips the two-character prefix for display. **[bin]**

#### Connecting

When the player connects to a server by name:

1. If the name starts with `"3 "` and the engine is not in 3D mode yet, it calls the launcher's "start Graal3D" callback with (name, host, port).
   - The launcher loads `GRAAL3DENGINE.DLL`, which removes that callback.
   - The launcher then reconnects with the same three values. **[bin]** That the reconnect call is `connectToGraalServer` is **[inf]**.
   - If the DLL is missing, the launcher has the text "You need to download Graal3D at … for playing this server!" **[bin][inf]**
2. With the DLL loaded:
   - The engine sets its 3D-mode flag (`isGraal3DMode()` returns it).
   - It calls the DLL's `connectTo3DServer(host, port + 1, localUdpPort)`.
   - It opens the Graal TCP connection to `host:port` exactly as for a 2D server: login id 5, then id 37 (see `protocol/v4x-clients.md`). **[bin]**
3. `localUdpPort` is the `fixedudpport` option (default 14899) when the `fixedport` option is on. Otherwise it is 0, meaning any free port. **[bin]**

#### State the DLL receives

The DLL pulls the login data from the engine through `getGraalLoginInfo()`. That function returns account name, nickname and password: the same values the launcher stores in the registry (`accountname`, `nickname`, `password`). **[bin]** No session key or cookie is passed. **[bin]**

#### Callbacks the DLL gives the engine

| Callback | What it does in the DLL |
|---|---|
| connectTo3DServer(host, port, localPort) | Creates the client `MissionGroup`. Opens the UDP port. Creates a `GameConnection` named `ServerConnection` and connects to `host:port`. |
| removeGraal3DConnection(reason) | Disconnects the Torque connection. |

| Callback | What it does in the DLL |
|---|---|
| sendGraal3DScriptEvent(name, params) | Queues a Graal event line for the 3D server (see "Graal events"). |
| sendGraal3DScriptEvents() | Flushes the queue. |
| sendGraal3DServerText(a, b, c, d) | Sends one text message over the big-data channel. No caller was found in the engine. |
| removeGraal3DDatablocks(), removeGraal3DMissionGroup() | Clean up on a level or server change. |
| getDatablockProperties(obj, list) | Lists datablock fields for GS2 scripts. |
| setGraalWindowLocked, getAllowCursorOff, getGraalPlayerScriptObject | Mouse lock and the script object for the local player. |

**[bin]**

When the Torque connection times out ("The 3D server connection has timed out"), the DLL reports it with `setGraal3DConnectionError`. The engine then also drops the TCP session. **[bin]**

## Torque connection

Except where marked, everything below is stock TGE 1.x. Every field is a BitStream bit field, written least-significant bit first. `u8`/`u32` are 8/32-bit fields; `intN` is an N-bit field; `flag` is one bit; `str` is TGE's Huffman-coded string (the character-frequency table is identical to TGE's). **[bin]**

### Out-of-band handshake

| Packet | Type | Layout |
|---|---|---|
| Connect challenge request | 26 (0x1A) | u8 type, u32 connectSequence |
| Connect challenge response | 30 | u8 type, u32 sequence, 4 × u32 address digest |
| Connect request | 32 | u8 type, u32 sequence, 4 × u32 digest, str `GameConnection`, then the request fields below |
| Connect accept | 36 | u8 type, u32 sequence, **class table**, u32 protocol version |
| Connect reject / disconnect | 34 / 38 | as in TGE |

**[bin]** The address digest is TGE's MD5 over the client address, the sequence and a random server key. **[bin][inf]**

### Connect request fields:

| Field | Value sent by this client |
|---|---|
| u32 net class group | 0 (game) **[inf]** |
| u32 class CRC | A per-build value. The stock code rejects a mismatch with `CHR_INVALID`, so a custom server should skip this check. |
| str game name | `Graal3D` |
| u32 current protocol | **19** |
| u32 minimum protocol | **19** |
| str join password | The player's **Graal password, in clear text** |
| u32 argc | 2 |
| str argv[0], argv[1] | Account name, nickname |

**[bin]**

- Rejection reasons the code knows: `CHR_GAME`, `CHR_PROTOCOL_LESS`, `CHR_PROTOCOL_GREATER`, `CHR_PROTOCOL`, `CHR_INVALID`, `CR_INVALID_ARGS` (more than 16 arguments). **[bin]**

- The accept must carry protocol version 19 exactly. 5.00r7 uses 24 instead (`protocol/v5x-clients.md`). **[bin]**

**Class table (Graal change).**

```
for each class:  flag 1, int12 type, int12 classId, str className
end:             flag 0
```

The server lists its own networked classes this way, inside connect accept. The type is 0 (object), 1 (datablock) or 2 (event). The client matches each name against its own classes and keeps a map in both directions. From then on, every class id on the wire is **12 bits, in the server's numbering**. A class the client doesn't know maps to "unknown" and is skipped. So a server can pick its own ids, as long as it sends the names. **[bin]**

#### Packet header and rates

- Header (stock TGE): flag 1, int1 (sequence parity), int9 packet sequence, int9 highest ack, int2 type (0 data, 1 ping, 2 ack), int3 ack byte count, then the ack mask. **[bin]**
- Then: flag (rate changed) → int10 update delay, int10 packet size; flag (max rate changed) → two int10s. **[bin]**
- Rate preferences:

| Pref | Default | Allowed range |
|---|---|---|
| `pref::Net::PacketRateToServer` | 32 | 8–32 |
| `pref::Net::PacketRateToClient` | 32 | 1–32 |
| `pref::Net::PacketSize` | 800 | 100–450 |

The update delay is 1024 / rate ms. So in practice the client runs at 32 packets/s, 450 bytes each. **[bin]**

#### Length prefixes (Graal change)

Every event, every ghost update and every datablock is sent as `int14 bitLength` + payload. The reader always jumps to the end of the payload, so an unknown or newer class can be skipped. **[bin]**

#### Server → client data packet

1. int32 last move ack.
2. flag → { flag → float7 damage flash; flag → float7 white-out }.
3. flag (control object). Then either:
    - flag (dirty) → u32 **server time**, int12 control-object ghost, the control object's packet data;
    - or 3 × F32 compression point.
4. flag (camera) → int12 camera ghost, flag → camera packet data.
5. flag → int8 field of view.
6. Events, then ghosts.

**[bin]** The u32 server time is a Graal addition. **[bin]**

#### Client → server data packet

1. flag (first person). **[inf]**
2. 3 × F32 position of the control object, then flag → int12 its ghost index.

3. flag (client sends its own state) → the control object's packet data. Otherwise: int32 first move index, int5 move count (up to 30), the moves.
4. flag → int8 field of view.
5. Events, then ghosts.

**[bin]** While the player controls an object, the client sends that object's state instead of moves, so position is client-reported, as in 2D Graal. **[inf]**

#### Events

- Unguaranteed events: flag 1, int12 class id, sized payload, … flag 0.
- Guaranteed-ordered events: flag 1, flag (next in sequence) else int7 sequence, int12 class id, sized payload, … flag 0.

**[bin]**

| Event | Direction | Note |
|---|---|---|
| SimDataBlockEvent | server → client | int12 (id − 3), int12 class id, int12 index, int13 total, sized datablock data (TGE uses 10-bit ids) |
| GhostAlwaysObjectEvent, NetStringEvent, ConnectionMessageEvent, PathManagerEvent, SimpleMessageEvent | either | Stock TGE |
| RemoteCommandEvent | either | int5 argc, tagged-string arguments. The client acts only on `MissionStartPhase1/2/3` (see below). |
| Sim2DAudioEvent, Sim3DAudioEvent, LightningStrikeEvent | server → client | Stock TGE |
| BigDataStartEvent, BigDataChunkEvent | either | Graal text channel (below) |
| TerrainChunkEvent | either | Graal terrain streaming (below) |
| TerrainRequestUpdatesEvent | client → server | 256 × u32 (one per terrain chunk) |
| RequestReloadEvent | client → server | Asks the server to resend an object |

**[bin]**

#### Ghosts (Graal changes)

1. flag (ghost data follows).
2. int4 **index width** (the number of bits of the highest index, sent as is; TGE sends width − 3).
3. For each ghost: flag 1, index, flag (delete). If not deleted: int12 class id (**sent on every update, not only the first**), sized update data.
4. flag 0 at the end.

**[bin]** New objects go into the client's `MissionGroup`. If the engine is still downloading files over TCP, they wait until the download is done. **[bin]**

Networked object classes, all stock TGE unless marked:

- **Shapes and players:** AIPlayer, Player, ShapeBase, StaticShape, Item, TSStatic, Shape3ds (Graal, 3DS models), Projectile, Debris, Splash.
- **Vehicles:** FlyingVehicle, HoverVehicle, WheeledVehicle, VehicleBlocker.
- **Cameras:** Camera, PathCamera, AdvancedCamera.
- **World:** InteriorInstance, PathedInterior, TerrainBlock, WaterBlock, Sky, Sun, Precipitation, Lightning.

- **Mission and markers:** GameBase, Marker, MissionArea, MissionMarker, SpawnSphere, WayPoint, Trigger, PhysicalZone, ScopeAlwaysShape, SimpleNetObject.
- **Effects:** AudioEmitter, ParticleEmitterNode, FractalTree, fxFoliageReplicator, fxGrassReplicator, fxShapeReplicator, fxShapeReplicatedStatic, fxLight, fxSunLight, volumeLight, twSurfaceReference.
- **Graal-only replicators:** DynamicGrass, DynamicGrassReplicator, DynamicShapeReplicator.

Datablock classes (stock TGE):

- **Shapes:** ShapeBaseData, ShapeBaseImageData, StaticShapeData, ItemData, PlayerData, TSShapeConstructor.
- **Cameras:** CameraData, PathCameraData, AdvancedCameraData.
- **Vehicles:** FlyingVehicleData, HoverVehicleData, WheeledVehicleData, WheeledVehicleSpring, WheeledVehicleTire.
- **Projectiles and effects:** ProjectileData, DebrisData, ExplosionData, SplashData, ParticleData, ParticleEmitterData, ParticleEmitterNodeData, PrecipitationData, LightningData, DecalData, fxLightData.
- **Audio:** AudioProfile, AudioDescription, AudioEnvironment, AudioSampleEnvironment.
- **Other:** GameBaseData, SimDataBlock, MissionMarkerData, PathedInteriorData, TriggerData.

**[bin]**

`ShapeBase` carries a Graal `chat` string: the text above a player's head, with `e_emoticon` handling. It is drawn by `GuiShapeNameHud` with a `chatprofile`. **[bin]** How the client's own chat reaches the 3D server was not found in the Torque code. It probably goes over the TCP connection as in 2D. **[inf]**

#### Mission start

The server drives the start with `commandToClient`. The client handles these steps in native code:

| Server sends | Client does | Client answers |
|---|---|---|
| `MissionStartPhase1` | Script event `onGraal3DLoginStep(1)` | `commandToServer('MissionStartPhase1Ack')` |
| `MissionStartPhase2` | `onGraal3DLoginStep(2)` | `MissionStartPhase2Ack` |
| `MissionStartPhase3` | Starts scene lighting, `onGraal3DLoginStep(3)`. When lighting is done: `$missionRunning = 1`, `onGraal3DLoginStep(4)`. | `MissionStartPhase3Ack` (after lighting) |
| First control object (in a data packet) | Switches the screen to the `GUIContainer3D` GUI, `onGraal3DLoginStep(5)` | – |

**[bin]** Unlike the TGE example scripts, the acks carry **no sequence number**, only the command tag. Every other `commandToClient` name is ignored. **[bin]**

#### Graal events and text (big-data channel)

Graal-specific data runs over two Graal events. **[bin]**

| Event | Layout |
|---|---|
| BigDataStartEvent | int16 transfer id, str `s1`, str `s2`, str `s3`, int24 total length, int3 compression (0 none, 1 zlib, 2 bzip2) |
| BigDataChunkEvent | int16 transfer id, int8 length (up to 191), the bytes |

The sender compresses with bzip2 over 4,095 bytes and with zlib over 49 bytes. It sends the start event and then up to 16 chunks per call. **[bin]**

**Server → client** (the only direction this DLL handles; its server-side handler is empty). When `s1` (a weapon name) is set, the data goes to that weapon as `onReceiveText(s2, s3, params)`. Otherwise `s2` selects the action:

| `s2` | Effect |
|---|---|
| `set` / `unset` | Set or clear the player flag in the data |
| `events` | One line per event: `s <flag>` sets a flag; `t <target>,<kind>,<action>,<params>` triggers `action<action>` on a weapon or GUI (`weapon`, `gui`) or an NPC (`npc`). Kind `rc` is RC chat (`onRCChat`); kind `torque` is ignored. |
| `addweapon` | Add weapon `s3`; the data holds a 5-byte Graal integer (a time stamp **[inf]**) |
| `weaponscript` | Script for weapon `s3` |
| `deleteweapon` | Remove weapon `s3` |
| `addnpc` | Script for the client-side object named `s3` |
| `setmaterials` | Terrain material list |
| `rpg` | `onRPGMessage(data)` |

**[bin]** So the 3D server, not the TCP gserver, delivers the client's GS2 weapons and flags in 3D mode. **[bin][inf]**

**Client → server.** The 2D engine's game events are queued as lines and sent as one message with `s1 = ""`, `s2 = "events"`, `s3 = ""`:

| Line | Comes from |
|---|---|
| `w <weapon>` | getweapon |
| `s <name>=<value>` | client-variable and flag sets |
| `t weapon,<name>,<params>` | serverside triggeraction from a weapon |

**[bin]**

#### Terrain streaming (Graal)

- `TerrainChunkEvent` carries: int4 chunk x, int4 chunk y, u32 stamp **[inf]**, int10 length, then a zlib-compressed 16 × 16 piece of heights and materials. 16 × 16 chunks make a 256 × 256 terrain. **[bin]**
- The client tells the server what it already has with `TerrainRequestUpdatesEvent` (256 × u32). It caches terrain under `levels3d/terrain/`. **[bin][inf]**
- `.ter` files are never requested over TCP. **[bin]**

### The TCP gserver connection in 3D mode

The Graal TCP connection stays up for the whole session. **[bin]**

| Traffic | In 3D mode |
|---|---|
| S→C 0 `LEVELBOARD`, 14 `PLAYERWARP`, 28 `FLAGSET`, 31 `FLAGDEL`, 49 `PLAYERWARP2`, 101 `BOARDPACKET` | Ignored. The level, warps and flags come from the 3D server. **[bin]** |
| Level requests at startup, getweapon, client-variable and flag sets, serverside triggers | Rerouted to the 3D server (see above). **[bin]** |
| Chat, PMs, the player list, file transfer, RC | Not touched by the 3D-mode switch, so presumably still TCP. **[inf]** |
| Missing 3D resources (shapes, interiors, textures, sounds) | Fetched with the engine's normal TCP file download, by lower-case file name (everything except `.dso`, `.cs` and `.ter`). **[bin]** |

### Files

| Kind | Notes |
|---|---|
| `.dts` shapes | TGE format, version 24 or older. Sequence import is supported. **[bin]** |
| `.dif` interiors | TGE format: resource version 44, interior version 0. **[bin]** |
| `.ter` terrain | TGE format, version 3 or older. Normally streamed, not downloaded. **[bin]** |
| `.md3` | Quake 3 models, version 15 (`_lower`, `_upper`, `_head` parts plus skins and an animation config). Graal addition. **[bin]** |
| `.3ds` | 3D Studio models (`Shape3ds`). Graal addition. **[bin]** |
| `.gft`, `.dml` | TGE font and material-list files. **[bin]** |
| Images | `.png`, `.jpg`, `.jp2`, `.jng`, `.mng`, `.gif`, `.bmp`, `.bm8`, `.tga`, `.dds`. **[bin]** |
| `.zip`, `.gpake` | Archives the resource manager opens. **[bin]** |
| `.mis` | Not used. The mission arrives over the network. **[bin][inf]** |
| `.dso`, `.cs` | Not used as TorqueScript; never downloaded. **[bin]** |
| `levels3d/gui/*.png` | Shipped GUI skins: `default*` and `guiblue_*` controls, player-list and PM-bubble images, `defaultnamehud.png`, `default3darrow.png`, the title screen. **[file]** |
| `levels3d/help/*.hfl` | Help pages (About, Links, Key shortcuts). **[file]** |
| `languages3D/language*.cs` | Nine languages, each a list of `$language::Name = "text";` lines. Neither binary names these files, so they are probably loaded by server GUI scripts. **[file][inf]** |

### Torque version

The DLL matches open TGE 1.x in:

- the out-of-band packet numbers and layouts;
- the packet header;
- the Huffman string table;
- the rate limits;
- the DTS, DIF and TER versions;
- its class list.

It has none of TGE 1.4's new lighting system. So it comes from the same TGE 1.2/1.3 line as the 3.x client. **[bin] [inf]**

Open TGE source (1.3 to 1.5) is a good base for the formats and the low-level net layer. A compatible client or server must add the Graal changes:

- the class table in connect accept, with 12-bit class ids;
- 14-bit length prefixes on events, ghosts and datablocks;
- a class id on every ghost update, and the raw index width;
- 12-bit datablock ids;
- the server time and client state in the game packets;
- the acks without a sequence number;
- the big-data and terrain events;
- GS2 in place of TorqueScript.

### Not done

- Exact `Player` and `ShapeBase` update layouts (Graal adds at least `chat` and a name colour).

- How the 3D server authenticates the player. The client sends the plain password in the connect request, and the TCP session is open at the same time.
- ~~How the per-build class CRC is computed.~~ It is not computed: the table at `0x101ba8b4` is static `{0xFFFFFFFF, 0, 0, 0}` and is only read (`torque/wire-by-build.md`).
- When the launcher shows "You need to download Graal3D".
- A capture from a real 2006 3D server, to check all of the above.

---

## `torque/engine-and-formats.md`

## The Torque engine in Graal: version, classes, 3D formats, rendering

Date: 2026-10-30. Tags as in `README.md`.

### Which Torque

- **3.04 Linux** says so itself: `getVersionNumber()` returns 1100, `getVersionString()` formats 1.1 as `"1.100"`, `getCompileTimeString()` is `"Jan  3 2004 at 22:32:08"`, the connect game string is TGE's default `"Torque Game Engine Demo"`, the home folder is `~/.garagegames/` (3.151 changed it to `~/.graal`), and source paths are TGE's (`game/gameConnection.cc`, `editor/missionAreaEditor.cc`). So 3.x is built on a TGE 1.1–1.2 era code base. **[bin]**
- **3.153 Mac** is the same engine for PowerPC/Carbon ("MacOS Torque Game Engine"), plus `AdvancedCamera`, `fxFoliageReplicator`, `twSurfaceReference`, `volumeLight` (community TGE resources of the time), a `Shape3ds` loader for 3D Studio models and an `Md3.cc` loader for Quake 3 player models (`_head.md3`, `_upper.md3`, `_lower.md3`). **[bin]**
- **4.04r2** `Graal3DEngine.dll` (built from `K:\src\graalv4\user\engine\…`) is still near-stock TGE: `Player`, `AIPlayer`, `Item`, `Projectile`, `Explosion`, `Debris`, `Splash`, `Precipitation`, `Lightning`, wheeled/flying/hover vehicles, `PathCamera`, `Trigger`, `PhysicalZone`, `InteriorInstance`, `TSStatic`, the `fx*` replicators and `fxLight`, `DecalManager`, and every matching `…Data` datablock. It imports 623 symbols from the 2D `GraalEngine.dll`. **[bin]**
- **5.12** `Graal3DEngine.dll` (Oct 2007) is one engine for 2D and 3D. It keeps Torque's core (console, `SimObject`/`SimSet`/`SimGroup`, `NetConnection`/`GameConnection`, ghosting, `SceneObject`, `TerrainBlock`, interiors, `TSShape` DTS, `Sky`, `Sun`, `WaterBlock`, `fxSunLight`, the world and terrain editors) but **drops the TGE game layer**: no datablocks, no `GameBase`/`ShapeBase`/`Player`/vehicles/projectiles/items, no `ScriptObject`, no audio emitters. Graal's own classes take their place (below). **[bin]**
- **4.20r6 Mac** (Sep 2006) already has the 5.12 model: `GameObject` and Graal's replicators, no datablocks, protocol 21. So the TGE game layer was dropped between 4.04r2 and 4.20r6. **[bin]** (`wire-by-build.md`)
- **5.24r3 onward** has no Torque 3D left (no `GameConnection`, `TerrainBlock`, `Interior` or DTS code); only the class name `GuiGraal3DCtrl` survives. **[bin]**

### The 5.12 3D object model

All script-visible through GS2 property and method tables. **[bin]**

| Class | What it is | Notable members (strings) |
|---|---|---|
| `GameObject` (a ghosted `SceneObject`) | every 3D thing in the world | `mountObject`, `unmountObject`, `getMountedObject…`, `getNodePosition/Transform`, `getI` `checkCollisionByBox`, `clipPolygons`, `getControllingClient`, `findimg`/`hideimg` (Graal `onAdd`, `onRemove`; `TGameObjectVar` / `TGameObjectAttr` sync script variables to clients ("fail yet") |
| `GameShapeDTS` | Torque DTS model | `playAnimation(slot, name, flags)` with flags 1 hold, 2 at end, 4 attack animation; `playA` `endpos, timescale, transitiontime)`; `pauseAnimation`, `stopAnimation`, `forceAnima` `DTSAniThread` objects; mount points and `collision-`/`los-` detail names; `onEndAnimation` |
| `GameShapeDIF` | Torque interior | `activateLight`, `deactivateLight` |
| `GameShape3DS` | 3DStudio model | collision hulls are meshes named `Col-…` (must be closed and convex); colours `3dscolorRGB` |

| Class | What it is | Notable members (strings) |
|---|---|---|
| `GameShapeFT` | 3D text from a TrueType font | FTGL-style vectoriser and tesselator |
| `GameShape` (base) | | `pixelshader`, `vertexshader` (GLSL through `glCreateShaderObjectARB`), fractal-tree field `texLeaf`) |
| `GameMovementInterpolate` / `…List` / `…Rigid` | how an object moves | `move3d`, `finishMoves`, `addRigidMove`, `applyImpulse`, `TMovementRigid.mass`, rigid-body `onLeaveLiquid` |
| `Camera` | the view | `cameramode` (`free`, `lookat`), `followattachedobject`, `lookatposition`, `lookvector`, `eul` |
| `DynamicGrass`, `DynamicGrassReplicator`, `DynamicShapeReplicator` | ground cover | terrain alignment, slope limits, allowed surfaces, per-grid object counts |
| `Sky`, `Sun`, `WaterBlock`, `fxSunLight`, `TerrainBlock` | stock Torque | three cloud layers, fog volumes, fog storms with server-synchronised time |

The 2D Graal player is still the player; 3D objects can carry ganis and show-images (`.gani`, `e:emoticon`).

### 3D resource formats

| Ext | Format | Where |
|---|---|---|
| `.dts` / `.dsq` | Torque shape / sequence (`TSShape`). 5.12 also rejects shapes that are too old ("not supported anymore"); 4.04r2 reads up to version 24 (v4 thread). | 3.x, 4.04r2, 5.12 |
| `.dif` | Torque interior (`InteriorResource`, with `ForceField` and `MirrorSubObject`) | all |
| `.ter` | Torque terrain file, read from `levels3d/terrain/` | 3.x, 4.04r2 |
| terrain chunks | 5.12 streams terrain from the server instead: `TerrainChunkEvent` ("sending/received terrain chunk (x, y): len n"), `TerrainRequestUpdatesEvent`, `heights` and `heights.info` names, "Loading terrain from terrain chunks." Terrain textures must be 256×256×24-bit. | 5.12 |
| `.3ds` | 3D Studio | 3.153 (`Shape3ds`), 4.04r2 (`Shape3ds`), 5.12 (`GameShape3DS`) |
| `.md3` | Quake 3 model, split head/upper/lower | 3.153 |
| `.mis` | Torque mission file | 3.x, 4.04r2 (5.12 has no `.mis` loader string and builds its `MissionGroup` from ghosts **[inf]**) |
| `.ml`, `.ifl` | material lists, image-file lists for animated textures | stock Torque |
| `.gft` | Torque cached GUI font (`%s%s_%d.gft`) | 3.x, 5.12 |
| `.dso` | compiled TorqueScript, see `scripting.md` | 3.x only |

3.153 also maps terrain textures to detail textures and footstep sound types in script (`addMaterialMapping`, see `scripting.md`). No `.dts`, `.dif`, `.ter` or `.mis` files ship with any Graal client: the 3D content always came from the server. **[file]**

### Rendering

- Stock Torque display devices in every build: `OpenGLDevice`, `D3DDevice` (Torque's OpenGL-to-Direct3D wrapper, `opengl2d3d.dll` / `glu2d3d.dll`) and `Voodoo2Device` (full-screen only). Minimum screen mode 640×480. **[bin]**
- 3.x on Linux uses SDL + OpenGL, on Mac AGL + DrawSprocket. 4.x and 5.12 on Windows: the Delphi launcher draws the 2D game with DirectDraw/Direct3D 7 and hosts the Torque canvas for 3D (`GuiGraal3DCtrl`, `runGraal3DEngine`, `resizeGraal3DWindow`, `toggleGraal3DFullScreen`). **[bin]**
- Under Wine 9 with Xvfb and Mesa (llvmpipe), the 5.12 launcher's 2D window runs. On switching to 3D the Torque side does start (`Activating the OpenGL display device`, Mesa llvmpipe, `Graal3D has been activated!`), but the launcher then faults re-initialising DirectDraw and stops calling `runGraal3DEngine`. Calling that export from an injected thread keeps Torque's networking running (enough for the full handshake, see `network.md`). A real GPU or a different Wine Direct3D setup is still needed to see the 3D view. **[run]**
- Scene lighting is Torque's (`SceneLighting`, lighting cached per mission), run as the last mission-load phase. **[bin]**

**`torque/network.md`**

## Torque networking in the Graal 3.x–5.12 clients

Date: 2026-10-30. Tags as in `README.md`.

### How a client gets to a 3D server

The server list marks a 3D server by a `"3 "` prefix on its name (the "Graal 3D" tab; see `protocol/v3x-clients.md` for the other prefixes). Choosing one runs the same routine in 3.153 (Mac, `0x39f148`), 4.04r2 (`GraalEngine.dll` `0x100be000`) and 5.12 (`Graal3DEngine.dll` `0x10199310`): **[bin]**

1. Strip the two-character prefix.
2. If the name had `"3 "`: remember **UDP port = listed port + 1**, set the "3D mode" flag, and start the Torque side.
    - 3.153: creates a `SimGroup` named `MissionGroup`, then `exec("main3D.cs")`. That script creates `GameConnection(ServerConnection)` and calls `connect()`; the engine supplies the stored address.
    - 4.04r2 and 5.12: first ask the launcher (`graal_setStartGraal3D` callback; a non-zero answer cancels), then call the launcher's `graal_setConnectTo3DServer` callback with host, port + 1 and the UDP option. In 4.04r2 that loads the separate `Graal3DEngine.dll`.
3. In **both** cases (2D or 3D) open the normal Graal TCP connection to host:listed port and log in as usual.

So a Graal3D server is two servers on one host: a Graal gserver on TCP *P* and a Torque server on UDP *P* + 1. The client's own UDP port is random unless `$pref::Graal::fixedport` is set, in which case it is `fixedudpport` (default 14899). **[bin][file]**

Seen live **[run]**: the real 5.12 client under Wine, told to join `"3 Seihai Test"` at the loopback address on `14800` (by calling its exported `connectToGraalServer(name, host, int port)` from an injected DLL, because no login server was available to list a 3D server). In the same millisecond it sent the Torque connect challenge to UDP 14801 and opened TCP 14800. The TCP side carried the ordinary 5.x login frame (`G3D14097`, account `guest`, then ids 37 and 23), the same bytes it sends to a 2D server.

### Torque packet framing

Stock TGE 1.x `NetInterface`. The first bit of a datagram picks the kind: **[bin]**

- bit 0 = 1: a data packet for an established `NetConnection` (Torque's notify protocol: sequence numbers, acks, then events, ghost updates and moves in a `BitStream`).
- bit 0 = 0: an "info" packet, byte-aligned. The first byte is the type:

| Type | Name | Direction | Body |
|---|---|---|---|
| `0x1A` | ConnectChallengeRequest | C → S | U32 connect sequence |
| `0x1C` | ConnectChallengeReject | S → C | (not handled by the Graal clients) |
| `0x1E` | ConnectChallengeResponse | S → C | U32 connect sequence, 4 × U32 address digest |
| `0x20` | ConnectRequest | C → S | U32 sequence, 4 × U32 digest, string class name (`GameConnection`), then the class's `writeConnectRequest` (below) |

| Type | Name | Direction | Body |
|---|---|---|---|
| `0x22` | ConnectReject | S→C | U32 sequence, string reason |
| `0x24` | ConnectAccept | S→C | U32 sequence, then (5.12) a class-name table, then `writeConnectAccept` (U32 protocol version) |
| `0x26` | Disconnect | both | U32 sequence, string reason |

Types below `0x17` go to the master-server/LAN query handler. All integers are little-endian.

Strings inside info packets are Torque `writeString`: one flag bit (1 = Huffman coded with Torque's fixed character-frequency tree, 0 = raw), a U8 length, then the codes or bytes. Info packets are a plain bit stream after the type byte, so fields after the first string are no longer byte-aligned. `torque/tools/tbits.py` reads and writes this format, including the Huffman tree. **[bin][run]**

#### The full handshake, live

Seen live **[run]** from the real 5.12 client under Wine, answered by `torque/tools/tq.py`. Timestamps are relative to the challenge request. The 3-second gap before the connect request is the injector's own wait before it starts running frames:

```
+0.000 C->S 1a 00040000                                  challenge request, sequence 0x400
+0.000 S->C 1e 00040000 00010203 04050607 08090a0b 0c0d0e0f   challenge response (our digest)
+3.009 C->S 20 00040000 00010203 ... 0c0d0e0f 1d6213fd ...   connect request, 71 bytes (decoded
below)
+3.009 S->C 24 00040000 00 30000000                      connect accept: seq, empty class table,
protocol 24
+3.043 C->S 05 00 00 04 41 b8 00 ...                     first data packet (seq 1)
         ... one 17-byte data packet a second, and a 4-byte ping about every 4.6 s ...
+18.02 C->S 26 00040000 00 00                            disconnect, sequence 0x400, empty reason
```

The client's own console (captured by hooking its `Con::printf` at `0x1000a220`) logs the same steps: `Sending Connect challenge Request`, `Got Connect challenge Response`, `Sending Connect Request`, `Connection established 19`, and at the end `Issuing Disconnect packet.` / `You have been disconnected from the game server.`

The connect request decodes to exactly:

| Field | Value |
|---|---|
| type, sequence, digest | `0x20`, `0x400`, our 16 digest bytes echoed back |
| class name | `GameConnection` (Huffman string) |
| net class group, class CRC | 0, `0xFFFFFFFF` |
| game string | `Graal3D` |
| current / minimum protocol | 24 / 24 |
| join password | `7Y3bdzjf`, the plain built-in guest password (see `protocol/v5v6-windows-unpacked.md`), so for a real account it is the account password in clear |
| argc, argv | 2: `guest` (the account), then an empty string (the nickname, unset here) |

No bits are left over, so the field list above is complete.

**Why the client disconnected.** The disconnect came the moment our lab TCP listener (which never answers the Graal login) closed the TCP side. The client then reports `You have been disconnected from the game`

`server.` and drops the Torque connection with `0x26`. So the 3D connection only lives as long as the Graal TCP connection. **[run]**

**How the live run got this far.** Under Wine the 5.12 launcher faults when it re-initialises DirectDraw on switching to 3D (`Error when loading and initializing DirectDraw: Access violation …` in `game_console.log`), and stops calling the engine's per-frame export `runGraal3DEngine`. The challenge request goes out from inside `connectToGraalServer`, but nothing processes the reply. The injector in `torque/tools/conn.c` calls `connectToGraalServer`, then calls `runGraal3DEngine` every 15 ms itself, which runs Torque's network processing and gets the client through the rest of the handshake. **[run]**

### The connect request by version

`NetConnection::writeConnectRequest` first writes the **net class group** (U32, 0 = game) and the **class CRC** for that group (U32). Then `GameConnection::writeConnectRequest` writes: **[bin]**

| Field | 3.04 Linux | 3.153 Mac | 4.04r2 | 4.20r6 Mac | 5.12 |
|---|---|---|---|---|---|
| game string | `Torque Game Engine Demo` | `Graal3D` | `Graal3D` | `Graal3D` | `Graal3D` |
| U32 current protocol | 9 | 19 | 19 | 21 | 24 |
| U32 minimum protocol | 9 | 19 | 19 | 21 | 24 |
| string join password | server password pref | a string the engine sets (`main3D.cs` sets none; not traced) | the account password, plain text | not traced | the account password, plain text |
| U32 argc (≤ 16), then argc strings | script args | engine-set args (not traced) | 2: account, nickname | not traced | 2: account, nickname (field layout unchanged from 4.04r2) |

The 3.153 column is from `0x122b18` (write) and `0x122c54` (server-side read, which ends in the stock script callback `onConnectRequest`); 4.20r6 from `mac420.c` (`wire-by-build.md`). The 4.04r2 column comes from the v4 thread (`protocol/v4x-clients.md`); the 5.12 one is from `Graal3DEngine.dll` `0x1004e950` (write) and `0x1004ea00` (server-side read). 5.12's server-side read rejects with:

- game string not `Graal3D` → `CHR_GAME` ("Only official Graal clients are allowed to connect.")
- client current < 24 → `CHR_PROTOCOL_LESS` ("Please update your Graal, the server is requiring a new Graal version!")
- client minimum > 24 → `CHR_PROTOCOL_GREATER` ("The server runs an older version of Graal, contact the admins to update it.")
- argc > 16 → `CR_INVALID_ARGS`

**The accept.** In stock TGE the accept body is one U32, the server's protocol. 5.12 inserts a **class-name table** in front of it, read by `0x1000ac20` before `readConnectAccept` (`0x1004e900`) runs: **[bin][run]**

```
repeat while readFlag() == 1:
    U12  class type    (must be < 2: 0 = net object, 1 = net event [inf])
    U12  server class id   (0..255)
    string class name  (Huffman string, e.g. "NetStringEvent")
U32  protocol          (must be exactly 24)
```

For each entry the client looks up its own class of that name, type and group, and records server id to local id (`0x103ae410`) and back (`0x103b0410`), 256 slots per group and type. Ghost and event class ids on the wire are then the **server's** ids. This replaces stock Torque's fixed class numbering, and it is why the class CRC no longer

matters. An accept with an empty table (a single 0 bit) plus protocol 24 is enough to reach "Connection established", as in the live run above. An accept without the table bit (stock layout) is read one bit off, gives protocol 12, and is silently rejected. **[run]**

The 5.12 client accepts **exactly 24**, 4.20r6 exactly 21, 4.04r2 and 3.153 exactly 19, 3.04 exactly 9; anything else gives `CHR_PROTOCOL`. 3.153 and 4.04r2 also have the class table, with three types (object, datablock, event); 4.20r6 has two, like 5.12; 3.04 has none (see `wire-by-build.md`).

**Class CRC.** Stock Torque compares the group and CRC and rejects a mismatch with `CHR_INVALID`. In 5.12 the CRC table at `0x1037754c` is static data `{0xFFFFFFFF, 0, 0, 0}` and nothing writes to it; reading it from the running client and the live connect request both gave group 0 and CRC `0xFFFFFFFF`. The class table in the accept (below) takes the CRC's place. **[bin][run]** 4.04r2's table at `0x101ba8b4` is the same static `{0xFFFFFFFF, 0, 0, 0}` with no writer (`wire-by-build.md`). (3.04, 3.153 and 4.20r6 read from a static table too; their values were not checked.)

**Encryption.** None beyond Torque's own framing. The join password travels in clear. **[bin]**

### After the connection is accepted

Data packets use stock Torque `ConnectionProtocol` headers: flag 1, connect-sequence bit, 9-bit sequence, 9-bit highest ack, 2-bit type (0 data, 1 ping, 2 ack), 3-bit ack byte count, then the ack mask. The live client's first data packet (sequence 1) carries the "max rate" flag with two 12-bit values, which read as 2080 and 368 with the stock field order. Every data packet after that is 17 bytes with no events. A ping follows whenever nothing has been acknowledged for about 4.6 s. **[run]**

Mission download follows Torque's `MissionStartPhaseN` / `MissionStartPhaseNAck` remote commands (`commandToServer` / `clientCmd`…): **[bin][file]**

| Build | Phases |
|---|---|
| 3.153, 4.04r2 (stock TGE) | 1 "Download Datablocks & Targets", 2 "Download Ghost Objects", 3 "Mission Lighting" |
| 5.12 | 1 "Download Ghost Objects", 2 "Mission Lighting" (no datablocks exist any more) |

In 3.153 these are script functions (decompiled in the project folder, `dec3153/scripts_missionDownload.cs`): phase 1 acks with `commandToServer('MissionStartPhase1Ack', %seq)`, phase 3 calls `lightScene("sceneLightingComplete")` and acks phase 3 when lighting finishes. The loading screen is a snowball rolling across `LoadingGui` (`missionDownload2.cs`). In 5.12 the same steps are native and raise the GS2 event `onGraal3DLoginStep`.

Other Torque events and objects the Graal clients register (5.12, from RTTI): `NetStringEvent`, `RemoteCommandEvent`, `ConnectionMessageEvent`, `GhostAlwaysObjectEvent`, `RequestReloadEvent`, `TerrainChunkEvent` and `TerrainRequestUpdatesEvent` (terrain is streamed, see `engine-and-formats.md`), and Graal's own `BigDataStartEvent` / `BigDataChunkEvent`, which carry Graal text and scripts over the Torque connection (described from 4.04r2 in `protocol/v4x-clients.md`). **[bin]**

### For a Seihai 3D server

The minimum to get a real 5.12 client connected, tested live: answer `0x1A` with `0x1E` (echo the sequence, any digest the server can later check). Accept a `0x20` whose class group is 0, CRC `0xFFFFFFFF`, game string `Graal3D` and protocol 24/24. Check the join password against the account. Reply `0x24` with the sequence, the class table

(every net object and event class the server will send, by name), a 0 bit, and protocol 24. Keep the Graal TCP connection on the port below alive, or the client drops the 3D one too. From there it is Torque's notify protocol, events and ghosting, which TGE 1.x and Torque3D source document. Use protocol 19 for 4.04r2 clients.

To repeat the live run: `wine-3d-lab/` in the project folder has `run3d.sh` (starts the listeners and the 5.12 client, then injects a DLL built from `conn.c` to join a fake 3D server and drive the engine's frames), `crc.c` (reads the CRC table from the running client), and the captures (`capture-512/`, `capture-512-full/`). Run it as `DLL=conn4 TQARGS=accept run3d.sh <name>`.

---

**`torque/README.md`**

## Graal on the Torque Game Engine (3.x to 5.12)

Date: 2026-10-30

From 2004 to 2007 the Graal client was built on GarageGames' **Torque Game Engine (TGE)**. Torque supplied the console, the TorqueScript VM, the GUI, the resource manager, the renderer, and (for 3D servers) the UDP netcode. Graal's own 2D game, the GS1/GS2 interpreters and the Graal TCP protocol were native C++ on top of it. This folder is the cross-version picture, written so that a later Seihai suite (launcher + legacy 2D client + Torque client) can be planned from it.

Tags: **[bin]** read from a client binary · **[file]** read from shipped data · **[run]** seen while running a real client under Wine · **[inf]** inferred.

- network.md: how a client joins a 3D server, the Torque UDP handshake byte by byte (run live end to end against the real 5.12 client), the connect request in each version, 5.12's class-name table in the accept, the mission download phases.
- scripting.md: the compiled TorqueScript (`.dso` v33) format with Graal's opcode table, the decoder in `tools/`, and what the 3.153 scripts do.
- wire-by-build.md: which builds are Torque, the handshake and class table per build (3.04, 3.153, 4.04r2, 4.20r6, 5.12), the networked class sets, the 5.12 game packets (no moves, client-reported position) and the 5.12 `GameObject` update.
- engine-and-formats.md: engine version, classes per build, 3D resource formats (DTS, DIF, TER, 3DS), terrain streaming, rendering devices.
- `tools/`: `dsodis.py` / `dsodec.py` (DSO disassembler and decompiler), `tbits.py` (Torque bit stream and Huffman strings), `tq.py` (a minimal Torque server that completes the 5.12 handshake), `conn.c` (the injector that makes a real 5.12 client join a 3D server and drives its frames under Wine).

Related notes elsewhere: `protocol/v3x-clients.md` (3.04 / 3.153 Graal TCP side), `protocol/v4x-clients.md` and `graal3d/torque-engine-4x.md` (4.x, including the 4.04r2 `Graal3DEngine.dll` read: BigData events, commands, protocol 19), `protocol/v5x-clients.md` (5.x TCP side), `graal3d/` (the unrelated February 2000 DirectX Graal3D).

### Lineage at a glance

| Build | Date | Shape | Torque part | 3D world | Game string / protocol | Script language for menus |
|---|---|---|---|---|---|---|
| 3.04 Linux | Jan 2004 | one C++ binary (SDL, OpenAL) | full TGE, `getVersionNumber` = 1100 ("1.100"), built "Jan 3 2004 22:32:08" | TGE classes present, no 3D scripts shipped | `Torque Game Engine Demo`, 9 / 9 | TorqueScript `.dso` |
| 3.153 Mac | Feb 2005 | one C++ binary (Carbon, PPC) | full TGE, with Graal's net-layer changes (class table, 12-bit class ids, 14-bit lengths, BigData and terrain events) | stock TGE world + `main3D.cs` client scripts | `Graal3D`, 19 / 19 | TorqueScript `.dso` |
| 4.04r2 Win | Mar 2006 | Delphi launcher + `GraalEngine.dll` (2D) + `Graal3DEngine.dll` (3D plug-in) | 2D engine keeps Torque base classes only; the 3D DLL is near-stock TGE (Player, vehicles, datablocks, fxFoliage) | stock TGE with datablocks | `Graal3D`, 19 (from the v4 thread) | GS2 / native |
| 4.20r6 Mac | Sep 2006 | one C++ binary (PPC) | Torque core + netcode, Graal game layer | `GameObject` model as in 5.12, no datablocks | `Graal3D`, 21 / 21 | GS2 / native |
| 5.12 Win | Oct 2007 | Delphi launcher + one engine DLL, still named `Graal3DEngine.dll` | Torque core + netcode, but the game layer is Graal's own | `GameObject` + `GameShapeDTS/DIF/3DS/FT`, no datablocks, no `Player`/`ShapeBase` | `Graal3D`, 24 / 24 | GS2 / native |
| 5.24r3 and later | 2010+ | engine linked into `Graal.exe` | Torque 3D gone (only a leftover `GuiGraal3DCtrl` name) | none | n/a | GS2 |
| 6.x | 2010–2014 | native C++ per platform | none (see `protocol/v6x-clients.md`) | none | n/a | GS2 |

All rows are **[bin]**, except the 3.153 script column (**[file]**). The 3.153 and 4.20r6 rows, and the per-build wire details, are in wire-by-build.md.

### What a Seihai suite would need, in order

1. **A launcher** that offers both clients. Graal itself did this from 4.x: the launcher shows the server list, and a server whose listed name starts with `"3 "` is a 3D server.
2. **A 3D server pair.** Every Graal3D server is a normal Graal TCP gserver on its listed port **plus** a Torque UDP server on **port + 1**. The TCP side carries the login, chat, GS2 and the player; the UDP side carries the 3D world. **[bin][run]**
3. **The Torque handshake**, which is small, fully known, and tested live against the real 5.12 client (see `network.md`). The class CRC that stock Torque checks is a constant `0xFFFFFFFF` in 5.12. Instead, the server's accept carries a table of class names and the ids it will use for them, so a Seihai server picks its own class ids and only needs Graal's class names. **[bin][run]**
4. **The 3D world.** Two choices: rebuild the 4.04r2 world (stock TGE datablocks, so an open TGE 1.x / Torque3D code base gets most of the way), or the 5.12 world (Graal's own `GameObject` model, scripted

from GS2). 5.12 is the last and most Graal-specific; 4.04r2 is the easiest to reproduce. No 3D server software or 3D level data is in the archives yet, so either way the content must be made or found.

5. **Per-version profiles.** A profile picks the game string and protocol (`Torque Game Engine Demo`/9, `Graal3D`/19, `Graal3D`/21, `Graal3D`/24), whether the accept carries the class table (every build from 3.153 on) and with how many types (3 up to 4.04r2, 2 from 4.20r6), the mission phases (3 in TGE builds, 2 in 5.12), and whether menus are TorqueScript (3.x) or GS2 (4.x+).

### What is in the project folder (not in this repo)

`graal-decompile/torque/` in the project files holds the extracted 3.153 Mac and 3.04 Linux trees, all 44 decompiled 3.153 scripts (`dec3153/`), the 5.12 `Graal3DEngine.dll` Ghidra export and string/RTTI dumps (`g3de/`), and the Wine lab (`wine-3d-lab/`) with the live capture.

---

## `torque/scripting.md`

## TorqueScript in Graal 3.x: the `.dso` format and the shipped scripts

Date: 2026-10-30. Tags as in `README.md`.

Graal 3.x keeps its menus, options, key binds, message boxes and the 3D mission-loading flow in TorqueScript. **[file]**

- 3.04 Linux ships 39 compiled scripts in the TGE starter layout (`common/client`, `common/ui`, `graaldata/client`, `graaldata/languages`, `graaldata/main.cs.dso`). It has a private-message window and a player list in script, and no 3D scripts.
- 3.153 Mac ships 44 compiled scripts (`scripts/`, `scripts2D/`, `scriptsgui/`) and 13 plain `.cs` files (prefs, key configs, nine language tables). It adds the 3D client scripts.
- From 4.x on, the menus are native or GS2 and no `.dso` files ship.

All 83 files decode with the table below.

### File format (version 33)

All integers little-endian. **[file]**

| Field | Notes |
|---|---|
| U32 version | 33 |
| U32 size, bytes | global string table (NUL-separated) |
| U32 count, count × F64 | global float table |
| U32 size, bytes | function string table |
| U32 count, count × F64 | function float table |
| U32 codeLength, U32 lineBreakPairCount | |
| code | `codeLength` words; each is one byte, or `0xFF` followed by a U32 |
| lineBreakPairCount × 2 × U32 | line/ip pairs |
| U32 identCount, then per ident: U32 string offset, U32 count, count × U32 ip | patches identifier words in the code with global-string entries |

Code inside a function body uses the function tables; code outside uses the global tables.

**Mac quirk:** the PPC build writes the F64 tables **big-endian** while everything else stays little-endian (1.0 is stored `3F F0 00 …`). The decoder detects this per file. The 3.04 Linux files are little-endian throughout.

**Unused parameters:** a function argument that the body never uses gets an ident offset just past the end of the global string table, so its name is lost. The decoder prints `%unusedN`.

### Graal's opcode table

The numbering differs from stock TGE 1.x: two unused opcodes follow `CREATE_OBJECT`, and one follows `CALLFUNC`. Worked out from the scripts (every file decodes cleanly with this table and no others) **[file]**:

| # | Opcode | # | Opcode | # | Opcode |
|---|---|---|---|---|---|
| 0 | FUNC_DECL | 28 | SHL | 56 | STR_TO_UINT |
| 1 | CREATE_OBJECT | 29 | AND | 57 | STR_TO_FLT |
| 2 | (unused) | 30 | OR | 58 | STR_TO_NONE |
| 3 | (unused) | 31 | ADD | 59 | FLT_TO_UINT |
| 4 | ADD_OBJECT | 32 | SUB | 60 | FLT_TO_STR |
| 5 | END_OBJECT | 33 | MUL | 61 | FLT_TO_NONE |
| 6 | JMPIFFNOT | 34 | DIV | 62 | UINT_TO_FLT |
| 7 | JMPIFNOT | 35 | NEG | 63 | UINT_TO_STR |
| 8 | JMPIFF | 36 | SETCURVAR | 64 | UINT_TO_NONE |
| 9 | JMPIF | 37 | SETCURVAR_CREATE | 65 | LOADIMMED_UINT |
| 10 | JMPIFNOT_NP | 38 | SETCURVAR_ARRAY | 66 | LOADIMMED_FLT |
| 11 | JMPIF_NP | 39 | SETCURVAR_ARRAY_CREATE | 67 | TAG_TO_STR |
| 12 | JMP | 40 | LOADVAR_UINT | 68 | LOADIMMED_STR |
| 13 | RETURN | 41 | LOADVAR_FLT | 69 | LOADIMMED_IDENT |
| 14 | CMPEQ | 42 | LOADVAR_STR | 70 | CALLFUNC_RESOLVE |
| 15 | CMPGR | 43 | SAVEVAR_UINT | 71 | CALLFUNC |
| 16 | CMPGE | 44 | SAVEVAR_FLT | 72 | (unused) |
| 17 | CMPLT | 45 | SAVEVAR_STR | 73 | ADVANCE_STR |
| 18 | CMPLE | 46 | SETCUROBJECT | 74 | ADVANCE_STR_APPENDCHAR |
| 19 | CMPNE | 47 | SETCUROBJECT_NEW | 75 | ADVANCE_STR_COMMA |
| 20 | XOR | 48 | SETCURFIELD | 76 | ADVANCE_STR_NUL |
| 21 | MOD | 49 | SETCURFIELD_ARRAY | 77 | REWIND_STR |
| 22 | BITAND | 50 | LOADFIELD_UINT | 78 | TERMINATE_REWIND_STR |
| 23 | BITOR | 51 | LOADFIELD_FLT | 79 | COMPARE_STR |
| 24 | NOT | 52 | LOADFIELD_STR | 80 | PUSH |
| 25 | NOTF | 53 | SAVEFIELD_UINT | 81 | PUSH_FRAME |
| 26 | ONESCOMPLEMENT | 54 | SAVEFIELD_FLT | 82 | BREAK |
| 27 | SHR | 55 | SAVEFIELD_STR | | |

Operands: `FUNC_DECL` name, namespace, package, hasBody, end ip, argc, argc arg idents; `CREATE_OBJECT` parent ident, isDatablock, fail ip; `ADD_OBJECT` and `END_OBJECT` one flag; jumps one target; `SETCURVAR`, `SETCURVAR_CREATE`, `SETCURFIELD`, `LOADIMMED_*`, `TAG_TO_STR`, `ADVANCE_STR_APPENDCHAR` one; `CALLFUNC(_RESOLVE)` name, namespace, call type (0 function, 1 method, 2 parent). Everything else has none. This is the same operand shape as stock TGE 1.x. Opcodes 2, 3 and 72 never occur in the shipped files, so what they were is unknown.

### The decoder

`tools/dsodis.py file.dso` prints a disassembly. `tools/dsodec.py file.dso` prints TorqueScript-like source: calls, assignments, `new` blocks with their fields, `@`/`SPC`/`TAB`/`NL` concatenation, array variables, and `&&`/`||`

chains. Other control flow stays as `if (!cond) goto Lnn;` with labels, and `?:` expressions are not rebuilt (they show as an empty `goto`). Run Python with `-I` on untrusted files.

### What the 3.153 scripts do

The decompiled sources are in the project folder (`graal-decompile/torque/dec3153/`, and `dec304/` for 3.04). 3.04's key map is the same as 3.153's 2D map, under the name `moveMap`, with F3 bound in the map instead of globally. Highlights **[file]**:

- **Start-up.** `main.cs` sets log mode 6, opens the console, runs `main2D.cs` and writes default prefs (`$pref::Graal::fixedudpport = 14899`, `$pref::Graal::defaultfontsize = 22`, OpenAL audio). `main2D.cs` (package `graal`) loads the GUI files and key binds and shows `MainMenuGui2D`. The 2D game itself is a native GUI control named `Adventure` (`GuiGraalCtrl`), shown by `start2D()`.
- **Menus call into native code** through `Adventure.*`: `setNickName`, `setAccountName`, `setPassWord`, `setLanguage`, `openServerList`, `startOfflineMode`, `selectServer`, `serverDirectConnect` (the "direct connect" box on the server list), `connectToSelectedServer`, `retrieveAccountFields`. If `$serverstartconnect` is set and account and password are filled in, the main menu goes straight to the server list (this is how `-connect` / `graal://name` auto-join works).
- **2D keys** (`default.bind2D.cs`, map `moveMap2D`): arrows move, A/S/D actions (Space = `doS2`), Q, P, M, T = microphone, F5 load level, F7 player list, F8 server list. Global: F1 help, F2 console, F3 options, Alt+Enter full screen, Esc `escapeFromGame2D()`.
- **3D keys** (`default.bind3D.cs`, map `moveMap`): WASD and arrows move (each also calls the 2D move function, so the Graal player is driven too), mouse look with `getMouseAdjustAmount` (sensitivity clamped to 0.05–2.0, scaled by FOV/90), Space jump (`$mvTriggerCount2`), left mouse fire, right mouse first person, E free look.
- **3D start** (`main3D.cs`, package `graal3D`): runs prefs and canvas scripts, maps nine terrain textures to detail textures and footstep sounds (`addMaterialMapping("grass.jpg", "detail: terraindetail_grass", "sound: 1", …)`; sand, grass, patchy, cobbles, two desert, three snow), loads `PlayGui`, `LoadingGui`, the mission-download and connection scripts, sets net port 0, then `new GameConnection(ServerConnection)` and `connect()`.
- **Connection errors** (`serverConnection.cs`): maps every Torque reject code to a message box, including the Graal ones for `CHR_PROTOCOL_LESS` / `_GREATER`.
- **Chat over Torque** (`message.cs`): stock `clientCmdChatMessage` / `clientCmdServerMessage` with `addMessageCallback`, ending in native `onChatMessage` / `onServerMessage`.
- **Loading screen** (`missionDownload2.cs`): a snowball rolls across the screen and grows (`g3d_splashsnow1-3.png`), the player picture changes with progress (`g3d_playersnow1-5.png`), and the hair image marks the phase (`g3d_hairsnow1/3/4.png`). These are the Christmas 2004 3D assets in `scripts2D/`.

---

## `torque/wire-by-build.md`

## Torque on the wire, build by build (3.04 to 5.12)

Date: 2026-10-30. Tags as in `README.md`. Everything here is a static read of the client binaries (the 3.04 Linux and 3.153 Mac decompiles, the 4.04r2 and 5.12 `Graal3DEngine.dll`, the 4.20r6 Mac client). Nothing new was run. Addresses are for orientation; the decompiles stay in the project folder.

This note fills the gaps the other Torque notes left open: the 3.153 and 4.20r6 columns, the class CRC in 4.04r2, the 5.12 game packets, and the 5.12 `GameObject` update. The 4.04r2 plug-in itself is described in full in `graal3d/torque-engine-4x.md`; this note only compares against it.

## Which builds are Torque

| Build | Torque? | 3D side | Read from |
|---|---|---|---|
| 1998 to 2.31 (Delphi) | no | none (Graal3D 2000 is a separate DirectX program) | `version-profiles/matrix/` |
| 3.04 Linux (Jan 2004) | yes, the whole client | stock TGE networking, no 3D scripts shipped | `lin304.c` |
| 3.153 Mac (Feb 2005) | yes, the whole client | Graal-changed TGE, TGE game layer (Player, datablocks) | `mac315.c`, `mac.rtti` |
| 4.04r2 Win (Mar 2006) | 2D engine on Torque base classes; 3D in `Graal3DEngine.dll` | same family as 3.153 | `g3d404.c` |
| 4.20r6 Mac (Sep 2006) | yes, one binary | Graal's own `GameObject` layer, no datablocks | `mac420.c` |
| 4.20r8 Win (Jan 2007) | 2D engine on Torque base classes | the upload has no `Graal3DEngine.dll`, so not readable | `eng420.c` |
| 5.12 Win (Oct 2007) | one engine DLL for 2D and 3D | `GameObject` layer | `g3de512.c` |
| 5.24r3 and later | no (only the `GuiGraal3DCtrl` name is left) | none | `torque/engine-and-formats.md` |

**[bin]**

## The handshake per build

| | 3.04 | 3.153 | 4.04r2 | 4.20r6 | 5.12 |
|---|---|---|---|---|---|
| Game string | `Torque Game Engine Demo` | `Graal3D` | `Graal3D` | `Graal3D` | `Graal3D` |
| Protocol (current = minimum, accept must match exactly) | 9 | **19** | 19 | **21** | 24 |
| Class table in the accept | no (stock) | **yes, 3 types** | yes, 3 types | **yes, 2 types** | yes, 2 types |
| Class ids on the wire | stock (bit size from the class count) | 12 bits | 12 bits | 12 bits | 12 bits |
| Class CRC in the connect request | stock table (value not read) | not read | **`0xFFFFFFFF`** | not read | `0xFFFFFFFF` |
| Server side of the connect request | stock script `onConnectRequest` | stock script `onConnectRequest` | native | native | native |

**[bin]**

- **3.153 is not "the same family as 3.04"** on the wire, as `README.md` had it. Its `GameConnection::writeConnectRequest` (`0x122b18`) writes `Graal3D` and 19 twice, and its accept reader (`0x122a74`) wants exactly 19. Its connect-accept writers (in `0x260058` and `0x260544`) call the class-table writer (`0x2c954`, three types: object, datablock, event) and its accept handler (`0x260608`) reads the table (`0x2ca48`) before `readConnectAccept`, exactly as 4.04r2 does. Its class-id map (`0x2cbfc` / `0x2cc60`) has the same "identity until a table has been read" rule. **[bin]**

- **3.04 has no Graal changes** in Torque's net layer. Its accept handler (`0x81f9090`) reads the sequence and goes straight to `readConnectAccept`, which only checks for protocol 9 (`0x810db60`). It has none of the Graal net classes (no `BigData`…, no `TerrainChunkEvent`). **[bin]**
- **4.04r2's class CRC** is a static `.data` table at `0x101ba8b4` holding `{0xFFFFFFFF, 0, 0, 0}`. The only two references to it are the reads in `NetConnection::writeConnectRequest` (`0x1012c260`) and `readConnectRequest` (`0x1012c2b0`), so nothing ever computes a CRC. This answers the "how the per-build class CRC is computed" item in `graal3d/torque-engine-4x.md`: it is not; group 0 always sends `0xFFFFFFFF`, as in 5.12. **[bin]**
- **4.20r6 Mac** writes `Graal3D` and 21 (`0x15`), wants exactly 21 in the accept, and its class-id map takes type < 2, like 5.12. **[bin]**
- The port: 3.153's `main3D.cs` path, 4.04r2 and 5.12 all connect to **listed port + 1** (`network.md`). The 28000 in `protocol/v3x-clients.md` is only the default that Torque's address parser fills in when a string has no port. **[bin]**

So the line is: stock TGE in 3.04; Graal's net-layer changes (class table, 12-bit class ids, 14-bit length prefixes, BigData and terrain events) already in 3.153; the TGE game layer and datablocks dropped between 4.04r2 (Feb 2006 token) and 4.20r6 (Jul 2006 token); protocol 19 → 21 → 24.

### The networked class sets

A Seihai server sends these names in the accept. Read from the static class registrations (name, group mask, type, event direction), and cross-checked against the `ConcreteClassRep<…>` RTTI list where there is one: in 5.12 and 4.04r2 the two lists match one for one. **[bin]**

| Type | 4.20r6 Mac |
|---|---|
| object(0) | `Camera`, `DynamicGrass`, `DynamicGrassReplicator`, `DynamicShapeReplicator`, `GameObject`, `SimpleNetObject`, `Sky`, `Sun`, `TerrainBlock`, `WaterBlo` `twSurfaceReference` |
| event(1) | `BigDataChunkEvent`, `BigDataStartEvent`, `ConnectionMessageEvent`, `GhostAlwaysObjectEvent`, `NetStringEvent`, `RemoteCommandEvent`, `Reques` server), `SimpleMessageEvent`, `TerrainChunkEvent`, `TerrainRequestUpdatesEvent` (client → server) |

Every other class registers with group mask 0 (not networked). The `GameShape…` classes (DTS, DIF, 3DS, FT) are not net classes: they ride inside a `GameObject` update (below). **[bin]**

4.04r2 has 117 class registrations, 98 of them networked: 49 objects, 34 datablocks and 15 events (the full list is in `graal3d/torque-engine-4x.md`; `Item`, `Sky`, `Sun` and `Path` are registered by string address, so a plain string search misses them). 3.153's RTTI has the same TGE game, datablock and Graal event classes (including `BigDataStartEvent`, `BigDataChunkEvent`, `TerrainChunkEvent`, `TerrainRequestUpdatesEvent`, `DynamicGrass`…); of 4.04r2's net and datablock classes only `AudioProfile` is missing from it. **[bin]**

### 5.12 game packets (the `GameObject` era)

These are the client's own `GameConnection::readPacket` (`0x1004f080`) and `writePacket` (`0x1004f360`). They differ from 4.04r2's (`graal3d/torque-engine-4x.md`) in three ways: no move ack, no damage flash or white-out, and no moves at all. 4.20r6's reader (`0x74d90`) has the same shape as 5.12's. **[bin]**

**Server → client**, after the stock packet header and rate fields:

1. flag (control object) → flag (update follows) → u32 **server time** (ms; the client keeps it as its synchronised clock, likely what `timevar3` reads **[inf]**), int12 ghost index of the control object (must resolve to a `GameObject`), the object's packet data (its vtable `+0xfc`). A first control object starts login step 5.
2. flag (camera) → int12 ghost index (a `Camera`), flag → the camera's packet data. No camera flag clears the camera. An unknown ghost logs "Received unknown camera object."
3. flag → int8 field of view.
4. Then stock `NetConnection` data: events, then ghosts.

**Client → server:**

1. flag: set when a float in the connection (`+0x45c`) is 0.0. What it holds was not traced. **[inf]**
2. 3 × F32: the control object's position (the translation column of its transform, or zeros without one), then flag → int12 ghost index of the control object, or of the object it is mounted on.
3. flag (the client controls that object) → the object's packet data (vtable `+0xf8`). There is no "else moves" branch any more.
4. flag → int8 field of view, clamped to 1–179.
5. Then events and ghosts.

So in 5.12 the client always reports its own position, and no Torque `Move` structure is ever sent. **[bin]**

**Events and ghosts** are as in 4.04r2: every event and every ghost update carries an int14 bit length, ghost updates carry a class id on every update, and the ghost index width is a raw int4. In 5.12 the class id reader is `0x1000e0b0` (12 bits, then the server-to-local map), the length wrappers are `0x100984b0` / `0x10098510` (events) and `0x1009c0f0` / `0x1009c160` (ghosts), and new ghosts are added to `MissionGroup` (`0x10099780`). **[bin]**

**Login steps** (`onGraal3DLoginStep`, the value starts at −1): 2 phase 1 "Download Ghost Objects", 3 phase 2 "Mission Lighting", 4 lighting done, 5 the first control object, 6 the TCP file downloads have finished and the `GraalControl3D` GUI is attached. 4.04r2 numbers them 1 to 5 over three phases. **[bin]**

### The 5.12 `GameObject` update

`GameObject::packUpdate` (`0x100511a0`) and `unpackUpdate` (`0x10051640`) agree on thirteen flag-guarded sections. The mask bits are the server's dirty bits; on the wire each section is just a flag. Field names are only given where the code makes them clear; the rest are marked. **[bin]**

| # | Dirty bits | Payload after the flag |
|---|---|---|
| 1 | `0x1` | string, the object's name (empty for `unknown_object`) |
| 2 | `0x2` or `0x200` | 3 × 32 bits (velocity **[inf]**) |
| 3 | `0x8` | string (a setting of the movement object) |
| 4 | `0x10` | string (also passed to the movement object) |
| 5 | `0x20` | transform: position, rotation, then 32 bits (the packing of the first two was not traced) |

| # | Dirty bits | Payload after the flag |
|---|---|---|
| 6 | `0x80` | string that creates or replaces the shape object (`shape` **[inf]**) |
| 7 | `0x40`, and a shape object exists | the shape object's own data |
| 8 | any of `0xF00`, and a movement object exists | int14 bit length, then the movement object's data |
| 9 | `0x1000` | string, the chat text (the reader handles `e:emoticon`) |
| 10 | always 0 from this DLL | (the reader takes a string and loads it as a script or image; what the real server sent here is unknown) |
| 11 | any of `0x3FC000` | the attribute list |
| 12 | `0x400000` | string, then 4 bytes of colour (RGBA, 0–255) (nickname and `nickcolor` **[inf]**) |
| 13 | `0x800000` | int16 of booleans: bits `0x01`, `0x04`, `0x08`, `0x10`, `0x20`, `0x40` are used |

The GS2 properties on `GameObject` are `alpha`, `attachedtoobject`, `attr`, `attrobjects`, `chat`, `collision`, `iseditorobject`, `liquidheight`, `liquidtype`, `movetype`, `nickcolor`, `nickname`, `scaledshapebox`, `shadow`, `shape`, `shapebox`, `showterrain`, `skin`, `solid`, `velocity` and `watercoverage`, so the unnamed strings and booleans are among these. Tying each section to its property needs the GS2 property dispatcher, which was not read. **[bin][inf]**

### For a Seihai Torque side

Pick the profile by protocol: 9 (3.04, stock TGE 1.x, no class table), 19 (3.153 and 4.04r2: class table with datablocks, TGE game classes, moves), 21 (4.20r6) and 24 (5.12): `GameObject` model, two-type class table, client-reported position. 21 and 24 share the packet shapes read above; the class sets differ by three names. The open items for a server are the `GameObject` property map, the rotation packing in section 5, and the movement and shape sub-object payloads.